

Systematic Review

A Systematic Survey of Smart Contract Fuzzing: Methods, Techniques, and Architectures for Ethereum and Beyond

Luis Alberto López Alvar ¹, Luis de la Torre ¹, Zehua Wang ² and Sebastián Dormido Canto ^{1,*}

¹ ETSI Informática, Universidad Nacional de Educación a Distancia, 28015 Madrid, Spain; llopez988@alumno.uned.es (L.A.L.A.); ldelatorre@dia.uned.es (L.d.l.T.)

² Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC V6T 1Z4, Canada; zwang@ece.ubc.ca

* Correspondence: sebas@dia.uned.es

Featured Application

This survey provides a structured reference for researchers and practitioners designing automated vulnerability-discovery pipelines for blockchain-based architectures, enabling systematic selection of smart contract fuzzing techniques based on architectural taxonomies and reproducibility criteria.

Abstract

Blockchain architectures increasingly rely on smart contracts as programmable execution components, yet the security challenges they introduce remain only partially addressed. Fuzz testing has emerged as one of the leading automated techniques for smart contract vulnerability discovery; however, a systematic treatment linking classical fuzzing concepts to the specific architectural constraints of smart contract execution environments has remained elusive. We conduct a large-scale systematic review of 258 publications collected from Scopus and Google Scholar. The review is structured around four research questions covering general fuzzing limitations, EVM execution constraints, cross-contract interaction challenges, and technique transferability. The contributions are fourfold. First, we develop a conceptual taxonomy establishing an explicit correspondence between classical and smart contract fuzzing. Second, we present an architectural taxonomy of representative fuzzers unified under a generalised waypoint architecture that exposes key feedback domains and composition gaps. Third, we provide a categorised technique review spanning coverage-guided, hybrid, and learning-based approaches. Fourth, we offer a reproducibility critique with a structured research roadmap. Coverage-guided greybox fuzzing emerges as the dominant paradigm (51.2% of the 121 SC fuzzer tools); learning-based approaches (machine learning, reinforcement learning, and LLMs) are a small but emerging class, with LLM-guided fuzzing the fastest-emerging subcategory by recency. Feedback-domain composition remains sparse: most surveyed fuzzers combine only a few of the eight identified feedback domains, and none approaches the full set, leaving several high-value multi-domain compositions unexplored. Open challenges include scalable stateful exploration, standardised benchmarks and oracles, and EVM-specific architectural optimisations. The complete categorised corpus, coding, and search strategy are openly available.



Academic Editor: Gianluca Lax

Received: 14 July 2026

Revised: 6 September 2026

Accepted: 10 September 2026

Published: 14 September 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons](https://creativecommons.org/licenses/by/4.0/)

[Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: smart contract; fuzzing; blockchain architecture; EVM; coverage-guided fuzzing; vulnerability detection; systematic review

1. Introduction

Blockchain technology, originally conceived as a mechanism for maintaining tamper-resistant, chronologically ordered records across a distributed network, has matured from early cryptocurrency experiments into a general-purpose infrastructure for decentralised applications. Bitcoin [1] demonstrated the viability of a peer-to-peer electronic cash system in 2008, and Ethereum [2] subsequently broadened the paradigm by introducing smart contracts: self-executing programs deployed on-chain that run deterministically in the Ethereum Virtual Machine (EVM). The concept of encoding contractual obligations in executable code dates back to Szabo [3], but it was Ethereum's Turing-complete execution environment that made programmable, composable financial logic a practical reality. A deployed Ethereum smart contract is identified by an address, may hold a balance, and exposes a set of callable functions with which external actors interact by submitting transactions that invoke these functions. The predominant development language is Solidity [4], although alternatives such as Vyper and Yul are also in use.

Unfortunately, the adoption of blockchain and smart contracts has also been accompanied by severe and costly attacks, often due to domain-specific security pitfalls in smart contract implementations. The 2016 DAO exploit drained approximately \$60 million in Ether by exploiting a reentrancy vulnerability; the 2017 Parity Multisig wallet bugs led to the permanent freezing of approximately \$150 million; and the DeFi ecosystem has suffered substantial cumulative losses from smart contract exploits [5]. As of 2025, over 60 million smart contracts have been deployed on Ethereum, collectively managing over \$100 billion in total value locked (TVL) in DeFi protocols alone. The economic stakes make automated vulnerability discovery not merely desirable but essential: a single undetected bug in a widely used DeFi protocol can cause losses in the hundreds of millions.

Recognising this risk, the research community, industry, and non-profit organisations have invested heavily in techniques for verifying and testing smart contract correctness [5–7]. These span static analysis (e.g., Slither [8]), which inspects the source or bytecode for known vulnerability patterns without executing the contract; symbolic and concolic execution (e.g., Oyente [9], Mythril [10]), which reason over path constraints to reach specific program states; and formal verification, which establishes correctness guarantees against a specification. Each technique occupies a different point in the trade-off between coverage, precision, and soundness, and they are increasingly used in combination rather than in isolation. Among these, fuzz testing (commonly known as fuzzing) has emerged as one of the most practical and effective automated approaches for vulnerability discovery. Originally introduced by Miller et al. [11] as a random input generation aimed at triggering anomalous program behaviours, fuzzing has evolved dramatically over three decades: modern fuzzers integrate coverage-guided greybox mutation (AFL [12], libFuzzer [13]), symbolic execution, machine learning (ML), and large language models (LLMs) to improve effectiveness. Nevertheless, applying fuzzing to smart contracts introduces challenges that have no direct parallel in classical software testing. These include the stateful, transactional execution model of the EVM; the complexity of multi-contract interactions; domain-specific vulnerability classes (reentrancy, integer overflow, access-control violations); and the absence of standardised benchmarks and oracles. Figure 1 sketches out this smart contract fuzzing loop at a high level, before the formal treatment in Section 3 is conducted. While the literature on smart contract analysis has grown substantially, a systematic treatment that links classical fuzzing theory to these domain-specific challenges, and organises the resulting body of work around a coherent formal and architectural framework, has remained elusive.

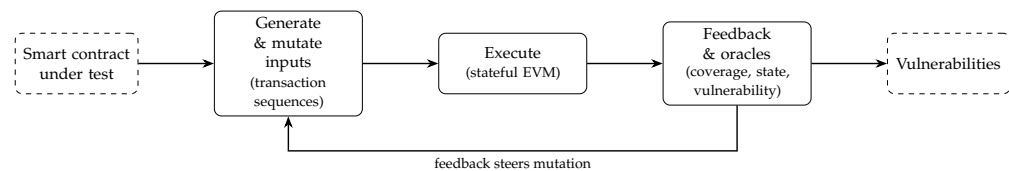


Figure 1. The smart contract fuzzing loop at a high level: a contract under test drives the generation and mutation of transaction sequences, which are executed on a stateful EVM; execution feedback and domain-specific oracles both surface vulnerabilities and steer further mutation. The loop runs on a local/embedded or forked EVM and therefore consumes compute rather than on-chain Ether.

This survey addresses this gap. The aim of this article is to conduct a systematic review of the smart contract fuzzing literature, providing a structured analysis of methods, techniques, and architectural designs under a coherent formal and taxonomic framework. The review is structured following the PRISMA 2020 framework [14] and addresses four research questions covering general fuzzing limitations, domain-level smart contract challenges, EVM-specific constraints, and technique transferability. The remainder of this article is organised as follows: Section 2 presents the review methodology, including the research questions, search strategy, study selection, and related works. Section 3 develops the taxonomy linking classical to smart contract fuzzing with formal models. Section 4 surveys smart contract fuzzer architectures. Section 5 reviews fuzzing techniques across six categories. Section 6 discusses the results, evaluation methodology, open challenges, and lessons learned. Section 7 concludes the survey.

2. Methodology

Systematic reviews aim to generate consensus and address specific problems, establishing a solid foundation for research [15,16]. This survey provides a structured analysis and classification of fuzzing approaches for Ethereum and EVM-compatible smart contracts, covering methods, techniques, and architectural designs. We examine the limitations and challenges documented in both the general fuzzing literature and the smart contract-specific literature, with particular attention paid to techniques that may transfer between the two domains. The scope is restricted to fuzz testing; other blockchain concerns (consensus mechanisms, network-layer scalability) are excluded unless they directly influence the fuzzing process. For this reason, the systematic review was conducted under the PRISMA 2020 framework, which provides a checklist ensuring transparency and reproducibility. Section 2.3 positions our work against prior surveys. As the first step in the methodology, the following research questions, focused on smart contract fuzzing, were established:

- RQ1 (General fuzzing): What are the current general limitations and challenges in fuzz testing?
- RQ2 (Domain-level): What limitations currently exist when fuzz testing smart contracts in general, such as challenges arising from the transactional execution model, stateful semantics, and domain-specific vulnerability classes that distinguish smart contracts from traditional software?
- RQ3 (Platform-level): Within Ethereum and EVM-compatible smart contracts specifically, what additional limitations do existing fuzzing techniques face due to EVM semantics (gas model, storage layout, cross-contract calls), the Solidity language, and the on-chain deployment environment?
- RQ4 (Transfer): What effective fuzzing techniques commonly employed in broader software testing are not yet widely used in the context of smart contracts and vice versa?

RQ2 and RQ3 are intentionally separated: RQ2 captures platform-agnostic smart contract challenges (e.g., transaction-sequence generation, oracle design for reentrancy or

financial properties), while RQ3 isolates limitations specific to Ethereum/EVM (e.g., the 256-bit stack architecture, gas metering, DELEGATECALL semantics, storage slot layout). This separation enables identification of which challenges will persist as fuzzing expands to other blockchain platforms (Solana, EOSIO, Hyperledger Fabric) and which are artefacts of the EVM.

Although several cross-cutting themes recur across levels, each is anchored to the research question carrying its primary evidentiary burden: statefulness, environment modelling, and oracle construction to RQ2 (domain); gas-aware execution and cross-contract interaction to RQ3 (platform); and RQ1 providing the general baseline and RQ4 the transfer view.

2.1. Search Strategy

To collect the relevant literature, we performed searches in two complementary databases: Scopus and Google Scholar. Using two databases with different indexing scopes follows the established systematic literature review (SLR) best practice and helps mitigate single-source bias. We first performed a preliminary reading to identify domain keywords (e.g., smart contract, Solidity, EVM) and fuzzing-related terms (e.g., fuzzing, fuzz testing, coverage-guided, property-based).

For Scopus, we used field-restricted queries (TITLE-ABS-KEY) and configured saved-search alerts covering smart contract fuzzing, classical fuzzing surveys, and their intersection, plus expanded queries for method- and vulnerability-focused tracking. For Google Scholar, we adapted the same conceptual queries using the allintitle operator. Scopus yielded 647 raw records and Google Scholar yielded 504 raw records (1151 combined). After removing 463 duplicates, 688 unique items remained. The saved-search alerts were monitored from early 2025, with 15 February 2026 taken as the final search date; no publication-year restriction was applied, so the corpus spanned the literature up to that date.

The exact query strings are reproduced in Supplementary S1. For Scopus we ran three field-restricted TITLE-ABS-KEY queries (S1 smart contract fuzzing; S2 fuzzing surveys; S3 their intersection) and three expanded queries (SA core-broad, SB methods/advanced, SC LLM/ML-assisted) to improve recall for specific method families. For Google Scholar we adapted the same concepts with the allintitle operator (G1–G3 baseline; GA domain, GB method-family, GC ML/LLM). Two databases were used deliberately rather than exhaustively: Scopus indexes the IEEE, ACM, Springer, and Elsevier venues in which this literature predominantly appears, so a separate pass over IEEE Xplore, the ACM Digital Library, or Web of Science would largely re-cover the same peer-reviewed records; Google Scholar complements Scopus by adding grey literature and preprints (e.g., arXiv) that Scopus does not index. Using one comprehensive index plus one broad grey-literature engine follows established dual-source SLR practice and mitigates single-source indexing bias. We note the reproducibility limits of the Google Scholar component: the allintitle operator restricts matching to title terms and therefore omits in-scope works whose titles do not contain the query terms; searches are run logged-out, and Scholar’s ranking, result count, and localisation are not deterministic across sessions, so exact result sets may not be reproducible. The Scopus baseline queries S1–S3 are reproducible from the strings provided; SA–SC are broader recall-oriented expansions of S1 (core-broad, method-family, and ML/LLM term variants) described in Supplementary S1, and all results returned by each Scholar query are screened, with the raw exports released so the record set is reproducible even where the live query is not.

2.2. Study Selection and Data Extraction

Application of the inclusion/exclusion criteria (IC1–IC3, EC1–EC3) to the 688 unique records yielded 258 included publications. The 430 excluded items were predominantly smart contract vulnerability-detection studies that did not employ fuzzing (EC2). Of the 258 included items, 230 originated from Scopus and 28 from Google Scholar. Each item included was assigned to exactly one of five categories: smart contract fuzzer tools ($n = 121$); web3 fuzzing surveys ($n = 31$); classical fuzzing surveys ($n = 96$); classical fuzzing techniques/tools ($n = 7$); and other ($n = 3$).

We distinguish two populations that are never conflated. The umbrella corpus of 258 publications records the full breadth of the PRISMA screening and is used only for the flow diagram and corpus-size statements; it is never used as a denominator. All analytical rates reported in this survey—technique shares, platform distribution, oracle usage, venue tiers, and publication-year growth—are computed on the fuzzer core of 121 smart contract fuzzer tools (118 unique) that satisfy the operational fuzzing-loop rule. Within this core, nested sub-samples support the architectural analysis: 33 representative tools, of which 29 carry sufficient public feedback-domain detail for the composition analysis, 31 support the technique–vulnerability cross-tabulation, and 22 are for the detailed head-to-head comparison; each subset membership is flagged per record in the released dataset.

Inclusion criteria:

- (IC1) Peer-reviewed conference paper, journal article, or workshop paper available in English;
- (IC2) Presents or evaluates a fuzzing technique, tool, or framework for smart contracts, or surveys fuzzing in the smart contract or classical domain;
- (IC3) Concerned with Ethereum, EVM-compatible, EOSIO, or WebAssembly-based smart contracts or their execution platform.

Exclusion criteria:

- (EC1) Duplicate records;
- (EC2) Works not related to fuzzing or smart contracts (e.g., pure formal verification or static analysis without a fuzzing component);
- (EC3) Works whose full text is not available in English (an English abstract alone is not sufficient).

The study selection process is shown in Figure 2. For each of the 121 fuzzer-core tools, we extracted: tool name; year and venue; primary technique category (CGF, PBF, HYB, ML, DIFF, Other); feedback type and coverage granularity; input model; target platform; oracle types; integration strategy; evaluation metrics; and open-source availability. These fields were coded for all 121 core tools; the 33 representative tools were additionally analysed at the architectural level (execution engine, feedback-domain composition, and integration strategy), a depth that requires public design detail, which is not available for every tool. Quality is assessed indirectly through the venue-tier analysis, the evaluation anti-pattern analysis (Section 6.3), and the reproducibility assessment (Section 6.3). Together with the inclusion gate (IC1–IC3 and the fuzzing-loop rule) and open-source availability, these form a lightweight quality-assessment framework, so we treat the PRISMA study-quality item as being addressed through it rather than not applicable.

For the venue-tier analysis, we classify publication venues into three tiers as a contextual visibility indicator, not a measure of methodological quality, using the CORE 2023 conference ranking and the CSRankings venue list (accessed February 2026): Tier 1 comprises CORE 2023 A* venues in software engineering, security, systems, and programming languages (e.g., ICSE, ESEC/FSE, ASE, ISSTA, IEEE S&P, USENIX Security, CCS, NDSS, OSDI, OOPSLA, and flagship journals such as TSE and *ACM Computing Surveys*); Tier 2

comprises other peer-reviewed venues ranked CORE A/B/C or indexed in Scopus/JCR; and Tier 3 comprises peer-reviewed workshops, companion tracks, and minor or regional venues. Standalone non-peer-reviewed preprints are excluded under IC1; a preprint of a work subsequently published is tiered by its published venue, so “preprint” is not itself a tier. The full per-venue tier assignment is released in `venue_tiers.csv` for direct verification.

The complete categorised corpus, the coding scheme, and the full search strategy are provided as Supplementary Materials and archived on Zenodo (DOI: 10.5281/zenodo.22539912).

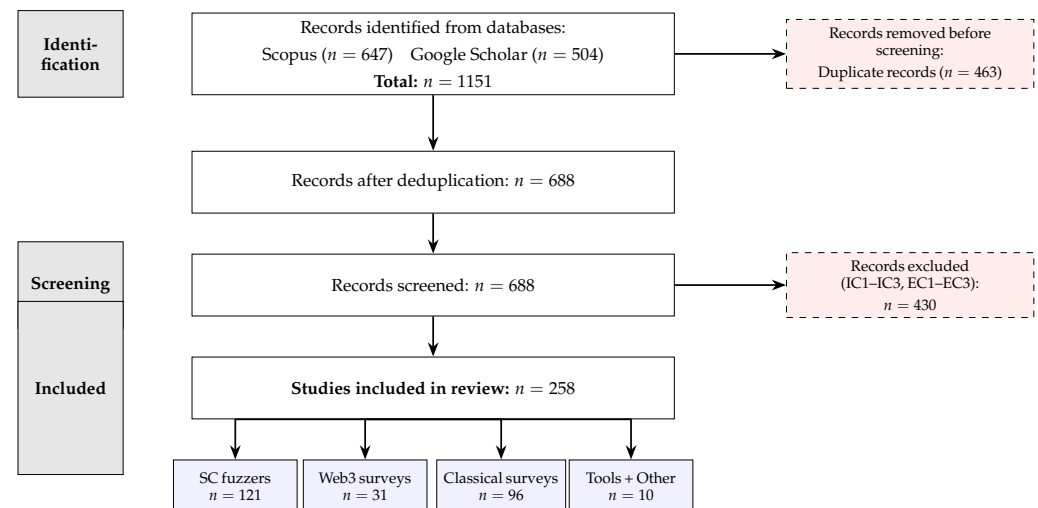


Figure 2. PRISMA 2020 flowchart. Records were identified from two databases (Scopus, Google Scholar). After deduplication and application of inclusion/exclusion criteria (IC1–IC3, EC1–EC3), 258 studies were included. Final categorisation: smart contract (SC) fuzzers ($n = 121$), Web3 fuzzing surveys ($n = 31$), classical fuzzing surveys ($n = 96$), classical tools and other ($n = 10$). Adapted from PRISMA 2020 guidelines [14].

2.3. Related Works

The literature on smart contract analysis has grown substantially over the past decade, with numerous surveys covering different facets of the problem. We organise the most relevant prior work into three groups: (i) smart contract security and vulnerability surveys, (ii) smart contract fuzzing surveys, and (iii) classical fuzzing surveys. Table 1 compares the most relevant surveys with our work along key dimensions.

Smart contract security and vulnerability surveys

Zhu et al. [5] (TSC 2024) summarise 27 types of exploited security vulnerabilities from 539 real-world incidents. Hejazi and Lashkari [6] (JNCA 2025) evaluate 256 smart contract analysis tools categorised by methodology; their scope is broad and does not provide a fuzzing architectural taxonomy or technique genealogy. Wei et al. [17] (ACM CSUR 2025) present a “Survey on Quality Assurance of Smart Contracts” that classifies 40 vulnerability types; fuzzing occupies a small part without an architecture survey. Dong et al. [18], Qian et al. [19], and Kushwaha et al. [20] survey vulnerability detection broadly.

Smart contract fuzzing surveys

Wu et al. [21] (ICSE 2024) provide the closest prior work: they compare state-of-the-art fuzzers by re-running them on a shared benchmark, exposing performance inconsistencies. Their contribution is primarily empirical; they do not survey architectural components, classify mutation operators, or provide a research roadmap. Our survey complements their empirical findings with architectural and taxonomic depth. Andrijasa et al. [22] survey multi-agent and RL-based fuzzing for smart contracts. Mnasri et al. [23] provide a comprehensive overview of Ethereum smart contract security testing. Tolmach et al. [7]

(ACM CSUR 2021) survey formal specification and verification of smart contracts and serve as our methodological model.

Classical fuzzing surveys

Manès et al. [24] (TSE 2021) introduce a unified model of the fuzzing process. Böhme, Cadar, and Roychoudhury [25] (IEEE Software 2021) distil open challenges for fuzzing and symbolic execution; their identification of semantic validity, stateful exploration, and environment modelling as fundamental barriers directly informs our analysis of smart contract fuzzing limitations (RQ1–RQ2). Beaman et al. [26] propose a refined fuzzer classification. Mallisery and Wu [27] (ACM CSUR 2023) systematise random-mutation, feedback-driven, coverage-guided, constraint-guided, seed-scheduling, and target-oriented strategies; their taxonomy informs our technique classification in Section 5. Bamohabbat Chafjiri et al. [28] review ML-based fuzzing across 91 studies, finding reproducibility barriers directly relevant to our ML/LLM analysis (Section 5.4). More recent surveys address coverage-guided grey-box fuzzing with deep learning [29], dictionary-based mutations [30], protocol fuzzing [31], kernel fuzzing [32], stateful system fuzzing [33], and ML/LLM-based fuzzing [34,35]. We draw on these to inform RQ1 and RQ4.

Table 1. Comparison of this survey with the most relevant prior works. Columns: SC = smart contract focus; Fuzz = fuzzing focus; RQs = explicit research questions; Arch = architecture analysis; Tables = classification/comparison tables; Quant = quantitative analysis; Corpus = corpus size. Marks follow predefined rules: ✓ = the dimension is present as an explicit, structured contribution (a dedicated section, taxonomy, or table); “partial” = addressed only incidentally or as a sub-part rather than a primary focus; blank = absent; “Corpus” reports the size and unit of analysis as stated by that survey.

Survey	SC	Fuzz	RQs	Arch	Tables	Quant	Corpus
Manès et al. [24]		✓		✓	✓	✓	–
Böhme et al. [25]		✓				✓	–
Beaman et al. [26]		✓			✓		–
Mallisery & Wu [27]		✓			✓	✓	–
Bamohabbat C. et al. [28]		✓			✓	✓	91
Tolmach et al. [7]	✓		✓		✓	✓	–
Wu et al. [21]	✓	✓			✓	✓	11 tools
Hejazi & L. [6]	✓	partial			✓	✓	256 tools
Andrijasa et al. [22]	✓	✓					–
Wei et al. [17]	✓	partial			✓	✓	14 tools
Mnasri et al. [23]	✓	partial					–
This work	✓	✓	✓	✓	✓	✓	258

Our contribution is best characterised not by an accumulation of features but by two substantive advances that, to our knowledge, prior surveys do not develop together: an explicit classical↔smart contract feedback-domain mapping, which grounds smart contract feedback in FuzzFactory’s domain-specific feedback model, and a generalised waypoint architecture that organises the surveyed fuzzers as instantiations of a common framework and exposes their feedback-composition gaps. We complement these with explicit research questions, classification and comparison tables, and a quantitative analysis of the corpus. In particular, we differentiate from: (a) Wei et al. [17], who address QA broadly with no formal fuzzing model or architecture analysis; (b) Wu et al. [21], whose contribution is empirical without formal modelling or a research roadmap; and (c) Hejazi and Lashkari [6], who evaluate 256 tools across all analysis paradigms without focusing on fuzzing or providing novel theoretical contributions.

To fix the terminology used throughout, Table 2 defines the core concepts; each is developed formally in Section 3.

Table 2. Glossary of core terms used in this survey.

Term	Definition
Fuzzing/fuzz testing/fuzzer	Automated testing that repeatedly executes a target on generated or mutated inputs to trigger faults or property violations; a fuzzer is a tool implementing this loop.
Feedback domain	A named dimension of execution feedback (keys, a value lattice, an aggregation operator) along which the fuzzer measures progress; branch coverage is one domain, storage-access or state novelty are others (Section 4.2).
Oracle (test-oracle)	A decision procedure that classifies an execution as buggy or vulnerable (e.g., a reentrancy or access-control oracle).
Oracle (price-oracle)	An on-chain data feed supplying external values (e.g., asset prices) to a contract; distinct from a test-oracle and a frequent manipulation target.
Coverage	A measure of what execution explored. Branch/edge counts control-flow decisions; storage tracks read/written slots; and state counts distinct blockchain states reached.
Seed	An input (here, a transaction sequence) kept in the corpus and reused as a basis for mutation.
Transaction sequence	The smart contract input unit: an ordered list $T = [tx_1, \dots, tx_k]$, with each transaction naming a callee, function selector, arguments, value, and environment.
Blackbox/greybox/whitebox	Fuzzing that uses no execution feedback (blackbox), lightweight feedback such as coverage (greybox), or full program analysis, e.g., symbolic execution (whitebox).
Snapshot	A checkpoint of the EVM state from which execution can fork/roll back without replaying the whole transaction history, enabling efficient stateful exploration.
Waypoint	An input saved because it makes monotonic progress along a domain-specific feedback dimension, not because it increases code coverage (FuzzFactory; Section 4.2).
ABI	Application Binary Interface: The contract's typed function/argument schema defining how calldata are encoded, i.e., the "grammar" of valid transactions.

3. A Taxonomy of Fuzzing: From Classical to Smart Contract Fuzzing

This section presents the formalisms and concepts used for fuzzing in the literature, and establishes a taxonomy that connects classical fuzzing with the smart contract fuzzing domain. The classification is organised into two broad categories: classical fuzzing (Section 3.1), which covers input–feedback models and guidance strategies applicable to

general software, and smart contract fuzzing (Section 3.2), which addresses the domain-specific input space, state model, and oracles required when fuzzing Ethereum or EVM-compatible contracts. This structure mirrors the contract-level vs. program-level distinction in formal verification surveys [7], and provides the conceptual foundation for the architectural survey (Section 4) and the techniques survey (Section 5). This section directly supports the answers to RQ1–RQ4.

3.1. Classical Fuzzing: Concepts and Input–Feedback Models

In classical fuzzing, the target program is executed with generated or mutated inputs; the goal is to trigger bugs, crashes, or violations of desired properties. The following concepts are central to the taxonomy:

- **Input space**—The set of all possible inputs to the program. For traditional software this may be byte streams (e.g., file or network input), structured data (e.g., XML, JSON), or protocol messages. Inputs can be generated from scratch (generation-based) or produced by mutating existing seeds (mutation-based).
- **Feedback**—Information observed from execution, used to guide the fuzzer. Common feedback includes: whether the program crashed or hung; code coverage (e.g., branch or edge coverage); and custom oracles (e.g., assertion failure, sanitiser report). Black-box fuzzers use no execution feedback; greybox fuzzers use lightweight feedback such as coverage; and whitebox approaches use full program analysis.
- **Guidance**—The strategy that determines which inputs are retained, prioritised, or mutated. Guidance may be random, coverage-guided (favour inputs that reach new code), grammar-based (respect input structure), or property-based (target specific invariants or violation conditions).

3.1.1. Input Space and Feedback in Classical Fuzzing

Coverage-guided greybox fuzzing (CGF) has become the dominant paradigm for vulnerability discovery in traditional software. The two most influential tools are AFL (American Fuzzy Lop) [12], which pioneered compile-time instrumentation and edge-coverage-guided mutation, and libFuzzer [13], which provides in-process, coverage-guided fuzzing integrated with LLVM sanitisers. Together they have inspired hundreds of descendants and have been adopted by major software projects (e.g., OSS-Fuzz [36], Google’s continuous fuzzing infrastructure). Honggfuzz [37] adds hardware-based coverage feedback and multi-threaded execution.

AFL internals and power scheduling.

AFL instruments the target binary at compile time (or via QEMU/Unicorn for binary-only targets) to collect an edge-coverage bitmap: a fixed-size array indexed by the hash of (source, destination) basic-block pairs. During fuzzing, AFL maintains a corpus of inputs, each tagged with the set of edges it covers. An input is retained if it covers at least one new edge. The power schedule determines how many mutations are applied to each seed: AFL’s default schedule assigns more “energy” (mutation rounds) to seeds that are smaller, faster to execute, and cover rare edges. Subsequent work (AFLFast [38], MOpt [39], FairFuzz [40]) refines the power schedule using Markov-chain models, particle-swarm optimisation of mutation operator distributions, and fairness-based prioritisation of rare branches. These scheduling innovations are directly transferable to smart contract fuzzing, where sFuzz [41] adopts AFL-style power scheduling and MuFuzz [42] introduces sequence-level energy assignment.

libFuzzer and sanitiser integration.

libFuzzer [13] operates in-process: the target function is compiled as a library and called repeatedly in the same address space, avoiding fork overhead. It integrates tightly with LLVM sanitisers (AddressSanitizer, MemorySanitizer, UndefinedBehaviourSanitizer), which serve as powerful oracles beyond crash detection. This in-process model is analogous to smart contract fuzzers that embed an EVM emulator (e.g., Echidna [43], ItyFuzz [44]) rather than interacting with a separate blockchain node, achieving much higher throughput.

Domain-specific feedback and waypoints.

Padhye et al. [45] (OOPSLA 2019) introduce FuzzFactory, a classical fuzzing framework included in our corpus under classical fuzzing tools, for developing domain-specific fuzzing applications by extending the CGF algorithm with waypoints: intermediate inputs that make progress toward domain-specific objectives beyond code coverage, without requiring changes to mutation or search heuristics. Users specify domain-specific feedback collection and aggregation; the framework then selectively saves waypoints that improve along these custom dimensions. FuzzFactory demonstrates that six distinct domain-specific fuzzing applications can be unified under a single framework and, crucially, composed to achieve emergent capabilities. Concretely, a waypoint is an input saved not because it adds code coverage but because it makes monotonic progress on a domain-specific feedback map $dsf: K \rightarrow V$ (feedback keys K , a partially ordered value set V , and a max/min aggregation recording the best value per key); an input is retained if it strictly improves the aggregate for at least one key in some active domain, and domains compose disjunctively (retain-if-waypoint-in-any). Its six sample applications each add one such domain—targeting, e.g., slow paths, input validity, and comparison-progress smoothing of magic-value branches; composing the validity domain with the comparison-smoothing domain discovers well-formed inputs that also clear hard equality guards, with each domain unblocking the other—the emergent capability. This waypoint concept is directly relevant to smart contract fuzzing, where domain-specific feedback such as storage-access patterns (sFuzz2.0 [46]), state novelty (ItyFuzz [44]), and oracle-specific signals go beyond branch coverage; indeed, ItyFuzz’s snapshot mechanism can be viewed as an instantiation of the waypoint pattern for stateful contract exploration (RQ4).

Systematic surveys and open challenges.

Böhme, Cadar, and Roychoudhury [25] reflect on the open problems of fuzzing, including how to fuzz stateful software and software with highly structured inputs. Building on their reflections and the challenge framing of Torres et al. [47], we organise three challenges that recur across fuzzing domains and are particularly acute for smart contracts: (i) *semantic validity*—ensuring that generated inputs are accepted by the program’s parser and reach deep logic; (ii) *stateful exploration*—effectively navigating systems whose behaviour depends on accumulated state; and (iii) *environment modelling*—faithfully simulating the program’s operating environment (files, network, hardware). All three challenges manifest acutely in smart contract fuzzing, where inputs must conform to the ABI format (semantic validity), execution is inherently stateful (state exploration), and the blockchain environment (gas, block context, other contracts) must be modelled (environment modelling). Mallisery and Wu [27] provide a comprehensive taxonomy of fuzzing methods—random-mutation, feedback-driven, coverage-guided, constraint-guided, seed-scheduling, and target-oriented—and demonstrate that no single strategy dominates; effectiveness depends on the combination of method, tool, and target. Their taxonomy informs our technique classification in Section 5.

Surveys of classical fuzzing [29,31] systematise the input space (e.g., for network protocol fuzzing, the input is message sequences with a structure defined by the protocol)

and the feedback mechanisms (coverage, crash, timeout). Beaman et al. [26] survey fuzzing vulnerability discovery techniques with a focus on automation and scalability, proposing a refined classification of fuzzers by generation method and guidance strategy, and identifying the need for domain-specific adaptations, a finding that motivates our focus on the smart contract-specific adaptations in Section 3.2. Dube et al. [30] conduct a systematic review of dictionary-based mutations in greybox fuzzing, finding that many tools rely on random-mutation operator distribution and that strategies for extracting dictionary tokens from the codebase are not yet optimised for highly structured input. Qiu et al. [29] survey coverage-guided greybox fuzzing with deep neural models, identifying challenges such as semantically invalid seeds, high computational overhead, and limited cross-domain adaptability, and pointing to LLM-powered fuzzing and neurosymbolic integration as promising directions. Bamohabbat Chafjiri et al. [28] review 91 studies on machine learning-based fuzzing and report that deep learning, reinforcement learning, and NLP techniques improve seed quality and mutation guidance, but that training cost, data dependency, and reproducibility remain obstacles to adoption in practice. These general limitations (RQ1) form the baseline against which smart contract fuzzing is compared.

3.1.2. Greybox, Grammar-Based, and Property-Based Fuzzing

Beyond pure coverage guidance, classical fuzzing includes grammar-based fuzzing, where inputs are generated according to a grammar or schema so that they are syntactically valid and more likely to reach deep code; property-based or invariant-based fuzzing (pioneered by QuickCheck [48]), where the goal is to violate user-defined or mined invariants; and symbolic or concolic hybrids, where symbolic execution (KLEE [49], SAGE [50]) helps to solve path constraints to generate inputs that reach specific branches. Driller [51] demonstrates the power of selectively combining coverage-guided fuzzing with concolic execution: when the fuzzer stalls, Driller invokes symbolic execution on difficult branches and feeds solutions back as seeds—a pattern adopted by smart contract hybrid fuzzers (ConFuzzius [47], TPH-Fuzz [52], WASAIUP [53]). Protocol fuzzing [31] and kernel fuzzing [32] surveys emphasise the importance of input structure and of feedback that goes beyond raw coverage (e.g., state machine coverage, protocol semantics). These techniques are transferable in principle to smart contract fuzzing (RQ4), where the “input” is a sequence of transactions and the “grammar” is the ABI and transaction format.

Following Manès et al. [24], a classical fuzzing campaign repeatedly selects a seed from a corpus, generates a mutated input, executes the target, extracts a feedback signal (typically an edge-coverage bitmap), and updates the corpus if the signal is novel. Grammar-based fuzzing constrains the mutator to produce structurally valid inputs; property-based fuzzing replaces the crash oracle with a user-defined predicate. The waypoint generalisation of this corpus-update rule (FuzzFactory [45]) underpins the unified architecture we develop in Section 4.2.

3.2. Smart Contract Fuzzing: Domain and Input Models

Smart contracts execute on a blockchain in a global, persistent state. Their behaviour is triggered by transactions: each transaction specifies a callee (contract address), a function selector and arguments (the calldata), and an optional value transfer. The state (storage, balances, and the effect of previous transactions) evolves with each execution. Thus, the input space for smart contract fuzzing is not a single byte stream but sequences of transactions together with the initial or evolving state. The key difference from classical fuzzing follows directly: rather than mutating a byte input to a stateless function, a smart contract fuzzer must synthesise ABI-conformant transaction sequences that drive this persistent state and judge the outcome with a domain-specific oracle over the whole execution trace

(Table 3). This shift instantiates all three challenges simultaneously: semantic validity (transactions must conform to the ABI and satisfy type/value constraints), stateful exploration (execution is inherently stateful, with each transaction modifying persistent storage), and environment modelling (the blockchain environment—gas model, block context, other deployed contracts—must be faithfully simulated). Each of these challenges has an analogue in classical fuzzing—grammar-based fuzzing for structured inputs, protocol and stateful fuzzing for accumulated state, and property-based fuzzing for non-crash oracles (Section 3.1.2)—but what is specific to smart contracts is that all three arise together, and by construction, in every campaign. These characteristics have profound implications for how fuzzing techniques from classical software must be adapted. We first introduce a formal model of the smart contract fuzzing problem (Section 3.2.1) and then discuss how it is instantiated in practice (Sections 3.2.2 and 3.2.3).

Table 3. Correspondence between the classical and smart contract fuzzing models.

Component	Classical	Smart Contract
Input space $\mathcal{I}$	Byte streams/structured data	Transaction sequences $T = [tx_1, \dots, tx_k]$
Execution	Stateless: $exec(i) \rightarrow r$	Stateful: $\delta(\sigma, tx) \rightarrow (\sigma', o)$
State	Implicit (process memory)	Explicit: $\sigma \in \Sigma$ (storage, balances, env)
Feedback ϕ	Edge bitmap, crash flag	Branch/storage-access cov., state diff
Oracle $\mathcal{B}$	Crash, sanitiser violation	Domain-specific: R, AC, Ar, G, Fin, CC
Mutation	Byte-level/grammar-level	Sequence-level: reorder, splice, type-aware
Scheduler	Power schedule (AFL)	Coverage + state novelty

3.2.1. Smart Contract Fuzzing Model

Smart contract fuzzing extends classical fuzzing along three dimensions (Table 3): (i) the input is a transaction sequence rather than a byte stream, where each transaction specifies a callee, function selector, arguments, Ether value, and environmental context; (ii) the execution is stateful—each transaction modifies persistent storage and balances, so the order and content of the sequence jointly determine the outcome; and (iii) the oracle is domain-specific and must inspect the full execution trace across the sequence, not merely a single crash or assertion failure. Coverage feedback in the smart contract setting can be gathered at branch level (used by most CGF tools), at storage-access level (tracking read/write patterns over contract storage slots, as introduced by sFuzz2.0 [46]) or at state level, recording the set of distinct blockchain states reached, as in ItyFuzz’s snapshot mechanism [44]. In a typical coverage-guided smart contract fuzzing loop, seeds are ABI-initialised, selected by a power schedule, mutated at the sequence level (by reordering, inserting, or splicing transactions), executed in an embedded EVM, and retained if they cover new code or trigger an oracle violation. Hybrid tools additionally invoke an SMT solver on unreached branches to generate targeted seeds, then resume greybox fuzzing with the enriched corpus.

Assessing how well such a fuzzer tests a contract requires more than code coverage. Jain et al. [54] show that code coverage and mutation score, the fraction of injected code mutants a test suite kills, are only weakly correlated: coverage shows that a line of code was executed, whereas killing a mutant on that line shows that the suite’s oracles can actually distinguish faulty behaviour. A campaign can therefore reach near-total coverage yet still miss injected faults. A second, smart contract-specific concern is oracle coverage, how many vulnerability classes the fuzzer’s detectors actually check, since a fuzzer that never instantiates, say, a reentrancy oracle cannot find reentrancy bugs regardless of coverage. Most tools nonetheless report coverage alone.

3.2.2. Transaction and State Models in Practice

At the contract level, smart contract fuzzing can be viewed as generating and mutating transaction sequences. Each element of the sequence is a transaction (or a block of transactions); the state after each transaction becomes the context for the next. Wu et al. [21] provide a critical analysis of state-of-the-art smart contract fuzzers and emphasise the need to explore stateful behaviours, multi-transaction sequences, and cross-contract interactions. Tools such as sFuzz [41], ConFuzz [55], and Midas [56] operate over transaction sequences and use coverage or custom oracles to guide exploration. Erinys [57] focuses on function invoke sequence generation; MuFuzz [42] introduces sequence-aware mutation and seed mask guidance; and CrossFuzz [58] and xFuzz target cross-contract vulnerability detection.

3.2.3. Oracles and Vulnerability Targets

Smart contract vulnerabilities are well-catalogued. Zhu et al. [5] summarise 27 types of exploited security vulnerabilities from 539 real-world incidents on Ethereum; Hejazi and Lashkari [6] evaluate 256 analysis tools including fuzzers. Complementary tools (Oyente [9], Mythril [10], Slither [8]) form the broader ecosystem and many hybrid fuzzers integrate with them. The common vulnerability classes are cross-tabulated against technique categories and representative fuzzers in the Discussion.

Three principal oracle designs cover the major vulnerability classes. A reentrancy oracle detects that a contract appears in the call stack while an external call from that same contract is still pending—including read-only reentrancy [59], where the reentrant call reads a stale state and induces incorrect downstream behaviour. An access-control oracle flags when a protected function is successfully invoked by an unauthorised sender (no revert). A financial property oracle, as in FinanceFuzz [60], checks invariant and equivalence properties such as token-supply conservation and balance equivalence. Most fuzzers combine multiple oracles with a disjunctive strategy (fire if any oracle triggers), which maximises recall at the cost of precision; the high false-positive rates reported in independent evaluations [21] are plausibly associated with this design. The technique-vs-vulnerability breakdown (Table 7) shows that reentrancy and arithmetic are targeted across all fuzzing-oriented categories, while financial properties and front-running are addressed by fewer than 10% of tools.

3.3. Summary

Section 3 has introduced a taxonomy linking classical fuzzing (input space, feedback, guidance) with smart contract fuzzing (transaction and state models, oracles, vulnerability targets). Table 3 makes the correspondence precise. The three challenges of semantic validity, stateful exploration, and environment modelling [25,47] manifest directly in the smart contract setting, and the multi-strategy perspective of Mallisery and Wu [27] motivates the technique category organisation of Section 5. The taxonomy clarifies how general limitations (RQ1) and smart contract-specific limitations (RQ2, RQ3) arise, and which techniques are transferable or underexplored (RQ4).

Having established what smart contract fuzzers must accomplish, we now turn to how they accomplish it: Section 4 surveys the architectural building blocks (execution engines, feedback mechanisms, scheduling strategies) that translate these concepts into working systems.

4. Survey on Smart Contract Fuzzer Architectures

This section surveys the architectures of smart contract fuzzers: the main components, how they are organised, and how data flow between them. It parallels the “Survey on Smart Contract Specifications” in formal verification surveys [7] by providing a structured

overview of how fuzzing is instantiated in the smart contract setting. We first describe the architectural components and data flow (Section 4.1), then present a generalised waypoint architecture that unifies all surveyed fuzzers under a common framework based on FuzzFactory's [45] domain-specific feedback model (Section 4.2) and classify fuzzers by design and feedback (Section 4.3). The classification draws on the categorised corpus of 121 smart contract fuzzers and the fuzzing surveys for web3 [5,6,21].

4.1. Architectural Components and Data Flow

A typical smart contract fuzzer can be decomposed into a small set of core components that interact in a well-defined data flow, illustrated in Figure 3. We present these components and then describe how they are combined in practice.

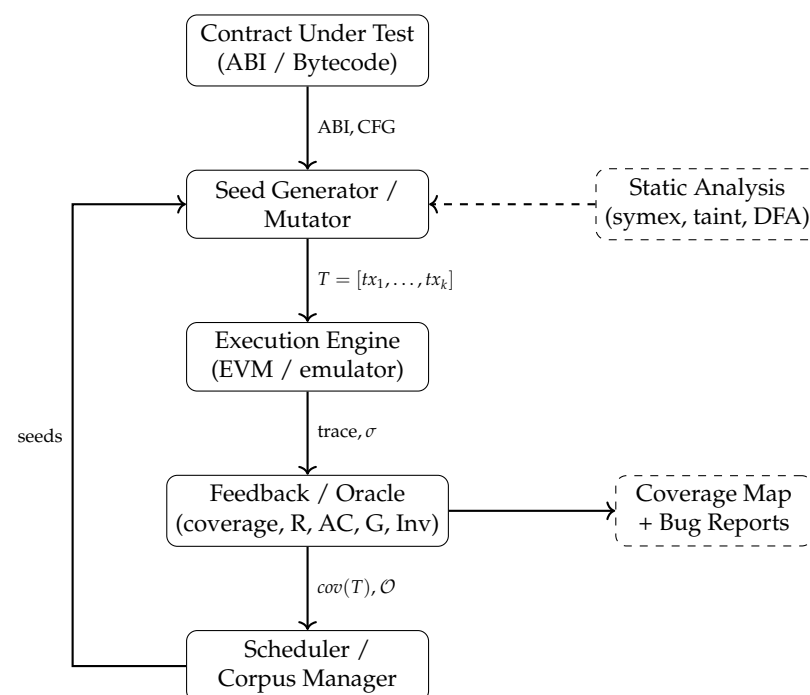


Figure 3. Generic architecture of a smart contract fuzzer, showing the five core components and data flow. Dashed connections indicate optional integration with static analysis or symbolic execution. In the generalised waypoint model (Section 4.2), the “Feedback/Oracle” component instantiates one or more domain-specific feedback maps $dsf_T : K \rightarrow V$, and the “Scheduler” implements the waypoint predicate $is_waypoint(T, S, D)$ over composed domains.

4.1.1. Input Representation and Generation

The input space for a smart contract fuzzer is the set of possible inputs that can be fed to the execution layer. In the smart contract setting, this is typically a transaction sequence: an ordered list of transactions, each specifying a callee (contract address), a function selector (or calldata), arguments, and an optional value. Some fuzzers also control the initial state (e.g., account balances, storage) or environment (block number, timestamp). The generation or mutation component produces or modifies these inputs. Generation may be random, grammar-based (using the contract ABI or a schema), or learning-based (e.g., LLM-generated sequences [61], reinforcement learning [62]). Mutation operates on existing seeds: it may change the order of transactions, the arguments or value of a call, or the length of the sequence. Tools such as sFuzz [41] and sFuzz2.0 [46] use coverage to guide which seeds to mutate; MuFuzz [42] introduces sequence-aware mutation and seed mask guidance; and ConFuzz [55] targets large-scale fuzz testing by scaling the input generation and execution pipeline.

4.1.2. Execution Engine Design

The execution layer runs the contract (or a model of it) on a given input. A critical architectural choice is how the execution engine is implemented; we identify three principal designs used across the surveyed tools:

- **Fork-based (external node):** The fuzzer communicates with a separate Ethereum node (e.g., Ganache, Hardhat Network, a private geth instance) via JSON-RPC. Each transaction is executed by this local node exactly as it would be on-chain, but on a private chain, so no mainnet gas or Ether is spent, and the node manages the state. ContractFuzzer [63] uses this approach, running against a private Ethereum network. Advantage: Highest fidelity (the full EVM semantics, including gas metering and precompiled contracts, are faithfully reproduced). Disadvantage: The IPC overhead of JSON-RPC limits throughput to hundreds of transactions per second, which is orders of magnitude slower than in-process execution.
- **In-process (embedded EVM):** The fuzzer embeds an EVM implementation as a library (e.g., Echidna [43] uses Hevm, a Haskell EVM implementation; ItyFuzz [44] uses a custom Rust-based EVM). Transaction execution occurs in the same process as the fuzzer, eliminating IPC overhead and enabling throughput of thousands to tens of thousands of transactions per second. This is analogous to libFuzzer's [13] in-process model for classical fuzzing (Section 3.1.1). Advantage: High throughput, direct access to execution traces, and easy instrumentation. Disadvantage: The embedded EVM may not implement all opcodes or precompiled contracts, introducing potential fidelity gaps.
- **Snapshot-based:** ItyFuzz [44] introduced snapshot management: the EVM state is periodically checkpointed, and the fuzzer can fork from any snapshot without replaying the entire transaction history. This avoids the cost of state reconstruction and enables efficient exploration of the state space from multiple starting points. sFuzz2.0 [46] uses a similar mechanism for storage-access tracking. Advantage: Enables efficient stateful exploration without full replay. Disadvantage: Snapshot management introduces memory overhead.

The choice of execution engine has a direct impact on the fuzzer's throughput, fidelity, and the types of vulnerabilities it can detect (e.g., gas-related bugs require accurate gas metering, which fork-based engines provide but simplified embedded EVMs may not). Modern tools increasingly favour in-process execution with snapshot support as the best compromise between throughput and fidelity.

4.1.3. Feedback and Scheduling

The feedback mechanism observes the execution and extracts the signals used to guide the fuzzer. Common feedback includes: (i) code coverage (e.g., branch or edge coverage, often obtained by instrumenting the EVM or bytecode); (ii) crash or revert; and (iii) custom oracles (e.g., reentrancy detection, gas anomalies, balance conservation). Coverage instrumentation can be performed at the source level (Solidity, before compilation), at the bytecode level (inserting probe instructions into EVM bytecode), or within the EVM interpreter itself (modifying the execution loop to record branch hits). Source-level instrumentation provides the highest granularity but requires access to source code; bytecode-level instrumentation (used by sFuzz [41] and SMARTIAN [64]) works on deployed contracts but may miss compiler-inserted code; EVM-level instrumentation (used by ItyFuzz) captures all executed code including library and proxy code. The scheduling or prioritisation component uses this feedback to decide which inputs to retain in the corpus and which to mutate next. Many fuzzers integrate with static analysis (e.g., CSAFuzzer [65] combines fuzzing with static analysis; SMARTIAN [64] uses static and dynamic data-flow

analyses) or with symbolic execution (e.g., ConFuzzius [47], TPH-Fuzz [52], WASAIUP [53]). Data flow from the input space through the execution layer to the feedback component, and feedback drives both generation (or mutation) and scheduling.

4.1.4. Implementation Characteristics

Beyond the architectural design, the implementation choices of fuzzers reveal important trends and practical constraints. We analysed the 33 representative tools in Table 4 and report the below observations.

Table 4. Classification of representative smart contract fuzzers by technique, feedback, target platform, and integration. CGF = coverage-guided greybox fuzzing; PBF = property-based fuzzing; HYB = hybrid (concolic/symbolic); ML = machine learning-assisted; DIFF = differential.

Tool	Year	Technique	Feedback/Oracle	Platform	Integration
Fuse [66]	2018	CGF	service-oriented arch.	Ethereum	standalone
ContractFuzzer [63]	2018	CGF	coverage + bug oracles	Ethereum	standalone
sFuzz [41]	2020	CGF	branch coverage	Ethereum/Solidity	standalone
Echidna [43]	2020	PBF + CGF	invariants + coverage	Ethereum/Solidity	standalone
Harvey [67]	2020	CGF	branch coverage	Ethereum	standalone
GasFuzzer [68]	2020	CGF	gas-oriented oracles	Ethereum	standalone
EthPloit [69]	2020	CGF	exploit generation	Ethereum	standalone
EOSFuzzer [70]	2020	Blackbox	bug oracles	EOSIO	standalone
ConFuzzius [47]	2021	HYB	coverage + data deps	Ethereum	symbolic exec
SMARTIAN [64]	2021	HYB	static+dynamic DFA	Ethereum	static analysis
Pluto [71]	2022	CGF	inter-contract oracles	Ethereum	static analysis
ReDefender [72]	2022	CGF	reentrancy oracle	Ethereum	standalone
ItyFuzz [44]	2023	CGF	snapshots + coverage	Ethereum	standalone
MagicMirror [73]	2023	CGF	combinat. testing	Ethereum	standalone
LDfuzz [74]	2023	CGF (directed)	branch coverage	Solidity	standalone
Effuzz [75]	2023	CGF (directed)	directed search	Ethereum	static analysis
WASAIUP [53]	2023	HYB	concolic + coverage	EOSIO/Wasm	symbolic exec
CrossFuzz [58]	2024	CGF	cross-contract oracles	Ethereum	static analysis
xFuzz [76]	2024	CGF + ML	cross-contract + ML	Ethereum	ML guidance
MuFuzz [42]	2024	CGF	seq-aware mutation	Ethereum	standalone
ConFuzz [55]	2024	CGF	coverage (large-scale)	Ethereum	standalone
FunFuzz [77]	2024	CGF	function-oriented	Ethereum	standalone
EvoFuzzer [78]	2024	CGF (evol.)	reentrancy oracle	Ethereum	standalone
Midas [56]	2024	CGF + DIFF	exploit mining	Ethereum	differential
sFuzz2.0 [46]	2024	CGF	storage-access patterns	Ethereum	standalone
Olympia [79]	2024	benchmark	fuzzer comparison	Solidity	benchmark
TPH-Fuzz [52]	2025	HYB	two-phase hybrid	Ethereum	symbolic exec
CSAFuzzer [65]	2025	HYB	static + fuzzing	Ethereum	static analysis
FinanceFuzz [60]	2025	PBF	financial properties	Ethereum	standalone
Vulseye [80]	2025	CGF (directed)	stateful directed	Ethereum	static analysis
DepFuzz [81]	2025	CGF	function dependence	Ethereum	static analysis
CrossGuard [82]	2025	ML + CGF	LLM + cross-contract	Ethereum	LLM
ACOFuzz [83]	2025	CGF (meta.)	ant colony + coverage	Ethereum	metaheuristic

Implementation languages. Fuzzers span a range of host languages: Haskell (Echidna), Rust (ItyFuzz, which builds on the revm EVM, and Foundry's Forge), C++ (sFuzz, built on the Aleth EVM), F#/.NET (SMARTIAN), and Go with a JavaScript/Node.js harness (ContractFuzzer); Python is common for orchestration in more recent tools, while Harvey remains proprietary. Rust is increasingly favoured for execution engines because it provides memory safety with near-C performance, enabling the in-process execution model without garbage-collection pauses.

EVM backend. The choice of EVM implementation determines fidelity and throughput (Section 4.1.2). Fork-based designs interact with Ganache or a geth node (e.g., ContractFuzzer), whereas more recent tools embed a custom or lightweight EVM (Hevm for Echidna, revm for ItyFuzz, py-evm for Python-based tools) or use the Hardhat Network. The shift from fork-based to embedded EVMs mirrors the AFL-to-libFuzzer evolution (Section 3.1) and is the single most impactful architectural decision for throughput.

Engineering effort. Reported implementation sizes vary widely—sFuzz, for instance, reports roughly 4300 lines of code [41], whereas tools that embed a full EVM and snapshot manager (e.g., ItyFuzz) are substantially larger. Larger codebases correlate with more sophisticated execution engines and richer oracle suites, but also with higher maintenance burden—a factor that helps explain why many academic tools become unmaintained not long after publication.

4.2. A Generalised Waypoint Architecture for Smart Contract Fuzzing

We observe that the architectural diversity among smart contract fuzzers can be unified under a single generalised framework. Padhye et al. [45] introduced FuzzFactory, which extends coverage-guided fuzzing with domain-specific feedback (DSF) and waypoints: inputs saved not because they improve code coverage, but because they make monotonic progress along a user-defined, domain-specific dimension. In this subsection, we use this as an open, extensible lens under which the smart contract fuzzers we survey can be described as instantiations of a generalised waypoint architecture, differing in which feedback domains they define and how they compose them. The eight domains are those we could ground in FuzzFactory’s DSF model and observe in the corpus, not a necessary-and-sufficient closed set; further domains—e.g., call-context novelty, protocol-level economic state, inter-contract topology, temporal properties, or environment feedback—may exist and can be added within the same framework. This unification reveals precisely which feedback dimensions are well-explored, which are missing, and how tools relate to one another.

4.2.1. Waypoint Domains for Smart Contracts

Under the FuzzFactory framework, each feedback dimension is specified as a domain: a set of feedback keys (e.g., branch IDs, storage slots, state hashes), a set of observed values, and an aggregation operator (typically max or min). A new transaction sequence is saved as a waypoint if it makes strict progress (i.e., changes the aggregate) for at least one key in at least one active domain. Domains compose via disjunction: a sequence is retained if it is a waypoint in any constituent domain. The domains are composable dimensions rather than a partition: they are neither mutually exclusive nor hierarchical, and a tool may instantiate several at once. This design guarantees that adding domains can only increase the set of retained sequences, never decrease it, and that progress is monotonic within each domain independently. This is a set-theoretic property of the disjunctive definition, not a guarantee of improved detection in practice, where finite time and memory, seed prioritisation, and scheduling overhead can offset it. For smart contracts, feedback hooks intercept EVM opcodes (JUMPI for branches, SLOAD/SSTORE for storage, CALL for inter-contract interaction) rather than LLVM IR constructs, which is the only adaptation required to apply FuzzFactory’s framework to this setting.

4.2.2. Domain Instantiations Across Smart Contract Fuzzers

We identify eight distinct feedback domains (Table 5) that, individually or in composition, recur across the fuzzers we surveyed: branch coverage (d_{br} , used by all CGF tools); storage-access patterns (d_{st} , sFuzz2.0); state novelty (d_{σ} , ItyFuzz); target distance (d_{dist} , LDFuzz, Vulseye); constraint proximity (d_{cmp} , ConFuzzius); data-flow coverage (d_{df} , SMARTIAN); property violation (d_{prop} , Echidna, FinanceFuzz); and oracle signal (d_{oracle} ,

ContractFuzzer). Composing domains can only increase the set of retained sequences, never decrease it [45] (a formal property of the disjunctive definition, not a claim of improved detection: finite time and memory budgets, seed prioritisation, and scheduling can offset it), yet composition is sparse: most tools employ only one or two of these domains and none approach the full eight.

Table 5. Eight waypoint feedback domains for smart contract fuzzers.

Domain	Keys/Values	Tools
d_{br} : Branch cov.	CFG branches/ $\{0, 1\}$, max	All CGF tools
d_{st} : Storage access	(slot,op)/ $\{0, 1\}$, max	sFuzz2.0
d_{σ} : State novelty	State hashes/ $\{0, 1\}$, max	ItyFuzz
d_{dist} : Distance	Targets/ $\mathbb{R}_{\geq 0}$, min	LDfuzz, Vulseye
d_{cmp} : Constraint prox.	Br-conditions/ $\mathbb{N}$, max	ConFuzzius
d_{df} : Data flow	(def,use)/ $\{0, 1\}$, max	SMARTIAN
d_{prop} : Property viol.	Invariant IDs/ $[0, 1]$, max	Echidna, FinanceFuzz
d_{oracle} : Oracle signal	Vuln IDs/ $\{0, 1\}$, max	ContractFuzzer, sFuzz

4.2.3. Tool-Level Domain Composition

We analysed the domain composition of all 33 representative fuzzers. Among the 29 with sufficient public domain information (Olympia, Harvey, GasFuzzer, and Fuse are excluded as they either serve as benchmarks or lack public feedback-domain detail), only a handful of tools compose as many as three domains (e.g., ItyFuzz: branch coverage, data flow, and state novelty); most use only two ($d_{br} + d_{oracle}$); and none approach the full eight. The released Feedback_Coverage field lets any reader re-derive this coding.

4.2.4. Gap Analysis: Missing Domain Compositions

Several structural gaps are evident: (1) storage access (d_{st} , used by only a few tools) is not combined with state novelty (d_{σ}), which would give deeper stateful coverage; (2) property violation (d_{prop}) is rarely combined with state-novelty feedback; (3) financial oracles lack directed target-distance feedback; and (4) no tool treats cross-contract reachability as a first-class waypoint domain distinct from constraint solving. More broadly, composition beyond two domains is rare, despite FuzzFactory showing that even two-domain composition yields emergent capabilities [45]. These constitute research directions (RQ4) require no change to the underlying mutation engine.

4.3. Tool Classification

Table 4 classifies 33 representative smart contract fuzzers by technique, feedback, target platform, and integration strategy. For additional per-tool dimensions—input model, coverage granularity, oracle set, hybrid/ML integration, and key innovation; see the detailed comparison in Table 6 (22 tools)—per-tool open-source availability is released in the dataset’s Availability field. The 33 representative tools were chosen by stratified purposive (maximum-variation) sampling across publication era (2018–2025) and technique family—spanning the families that employ feedback-domain composition (coverage-guided, hybrid, property-based, and ML-guided)—additionally requiring sufficient public architectural detail to code each tool from its primary source and preferring open-source tools where a stratum offered a choice; selection was not by citation count. The 33 (and the 29/31/22 sub-samples) are flagged with dedicated columns in the released dataset.

Table 6. Detailed comparison of representative smart contract fuzzers. “Seq” = transaction-sequence-level input; “Tx” = single-transaction input. Coverage granularity: Br = branch, Ed = edge, St = storage access. Oracle types: R = reentrancy, AC = access control, Ar = arithmetic, G = gas, Inv = user invariants, Fin = financial properties, CC = cross-contract. A checkmark (✓) indicates that the tool uses the technique.

Tool	Input	Cov.	Oracle	Hybrid	ML	Key Innovation
ContractFuzzer	Tx	–	R, AC, Ar, G			Among the earliest; ABI-based generation
sFuzz	Seq	Br	R, Ar, G			Adaptive power schedule for SCs
Echidna	Seq	Br	Inv			Property-based; multi-core; practical
Harvey	Seq	Br	R, Ar			Greybox; input prediction
ConFuzzius	Seq	Br	R, AC, Ar	✓		Data-dependency-aware hybrid
SMARTIAN	Seq	Br	R, AC, Ar	✓		Static + dynamic data-flow analysis
ItyFuzz	Seq	Ed	R, AC, Ar			Snapshot-based; on-chain state
MuFuzz	Seq	Br	R, AC, Ar			Seq-aware mutation; seed mask
CrossFuzz	Seq	Br	CC, R			Cross-contract vuln detection
xFuzz	Seq	Br	CC		✓	ML-guided cross-contract
FunFuzz	Seq	Br	R, Ar, G			Function-oriented; efficiency
sFuzz2.0	Seq	St	R, Ar			Storage-access-pattern guidance
Vulseye	Seq	Br	R, AC			Stateful directed greybox
ConFuzz	Seq	Br	R, Ar			Large-scale Ethereum fuzzing
FinanceFuzz	Seq	Prop	Fin			Financial property oracles
EvoFuzzer	Seq	Br	R			Evolutionary reentrancy detection
TPH-Fuzz	Seq	Br	R, AC, Ar	✓		Two-phase hybrid
CSAFuzzer	Seq	Br	R, AC, Ar	✓		Static analysis + fuzzing
CrossGuard	Seq	Br	CC		✓	Runtime-adaptive LLM fuzzing
ACOFuzz	Seq	Br	R, AC, Ar			Ant colony optimisation
DepFuzz	Seq	Br	R, AC			Function dependence guidance
Midas	Seq	Br	exploit			Exploit mining + diff. analysis

5. Smart Contract Fuzzing Techniques

We survey six technique categories represented in the 121 fuzzer-core tools. Each tool is assigned a single mutually exclusive primary category; each tool’s full mechanism set (beyond its single primary category) is recorded in the released extraction (mechanisms_present column), so no multi-mechanism information is lost.

5.1. Coverage-Guided (Greybox) Fuzzing

Coverage-guided fuzzing (CGF), the dominant paradigm (51.2% of tools), instruments the EVM to collect branch (or edge) coverage and guides mutation toward underexplored code. The line of work opened with ContractFuzzer [63] (ASE 2018), among the earliest SC fuzzers, which combined ABI-based generation with seven bug oracles. Throughput and scheduling improved rapidly thereafter; sFuzz [41] (ICSE 2020) added AFL-style power scheduling, achieving broader branch coverage than ContractFuzzer at roughly two orders of magnitude higher throughput, while Echidna [43] (ISSTA 2020) introduced property-based CGF with user-defined Solidity invariants and is widely used in practice. Later tools target specific weaknesses of pure coverage guidance, Harvey [67] (ESEC/FSE 2020) uses input prediction to bypass hard branches. Sequence-level variants include MuFuzz [42] (seed-mask mutation), MagicMirror [73] (combinatorial testing), and ConFuzz [55] (large-scale parallel fuzzing). Directed CGF tools (LDfuzz [74], Vulseye [80]) steer exploration via distance metrics. sFuzz2.0 [46] extends branch coverage with storage-access-pattern feedback, reporting 20.49% more detected vulnerabilities and 37.53% more branch coverage than sFuzz. Wu et al. [21] show that no single CGF tool dominates across benchmarks; sFuzz and Echidna are the standard baselines. A key challenge is that coverage plateaus quickly, and the remaining code often requires specific argument values or long transaction sequences that only hybrid or directed approaches can reach.

5.2. Property-Based and Invariant Fuzzing

Property-based fuzzing (PBF) (3.3% of tools) aims to violate user-defined invariants rather than merely trigger crashes. Echidna [43] is the canonical example: users write `echidna_*` boolean predicates and Echidna searches for a transaction sequence that falsifies one. FinanceFuzz [60] extends the idea of DeFi-specific invariant and equivalence properties such as token-supply conservation and balance equivalence across equivalent transaction sequences. Invariants can be user-supplied (Echidna, Foundry), mined from execution traces, or generated automatically. SmartACE [84] bridges PBF with bounded model checking for combined “fuzz-then-verify” workflows. The main barrier to adoption is the specification effort, which LLM-assisted invariant generation may help to lower.

5.3. Symbolic and Concolic Fuzzing

Hybrid symbolic and concolic fuzzers (24.0%) combine concrete execution with symbolic constraint solving to reach branches that random mutation cannot. ConFuzzius [47] (EuroS&P 2021) uses demand-driven SMT solving (Z3) to pass hard branches, invoking the solver only when greybox progress stalls. SMARTIAN [64] (ASE 2021) adds static and dynamic data-flow analysis to order transactions and guide mutation. Although greybox rather than concolic, ItyFuzz [44] (ISSTA 2023) is often discussed alongside hybrid tools for its snapshot-based state management ($O(1)$ state restoration vs. $O(k)$ replay), which enables efficient exploration of stateful contracts that prior tools could not reach. Later designs focus on controlling solver costs. TPH-Fuzz [52] uses a two-phase design: broad CGF then targeted symbolic. WASAIUP [53] brings demand-driven concolic fuzzing to EOSIO/Wasm contracts. Ding et al. [85] extend hybrid fuzzing to DeFi protocol-level vulnerabilities via context-sensitive concolic verification. The main limitations are inherited from symbolic execution: path explosion, intractable cryptographic constraints and high cost for long transaction sequences.

5.4. Learning-Based Fuzzing (ML, RL, LLM)

Learning-based fuzzers (13 of 121, 10.7%) use a learned model within the fuzzing loop as the seed/mutation generator, the transaction-selection policy, or the vulnerability

oracle applied to executed inputs. They comprise three model classes: classical machine learning (7), reinforcement learning (3), and large language models (3), released per-tool in the dataset's `learning_subclass/learning_mechanism` fields. Static ML/DL classifiers that label a contract without generating or executing inputs are excluded from the core under EC2. Among LLM-guided tools, Sun et al. [61] (Autom. Softw. Eng. 2024) use chain-based LLMs for adversarial seed generation, reporting an average instruction-coverage gain of $\approx 3\%$ over a single-step LLM baseline, and CrossGuard [82] queries an LLM at runtime for cross-contract mutation hints; among reinforcement-learning tools, Su et al. [62] model fuzzing as a Markov decision process to learn a transaction-selection policy. Reproducibility in this category depends on whether the underlying model is open or closed: closed- or API-served models are exposed to silent model drift and prompt sensitivity, so a campaign may not be re-runnable even when its code is released; Bamohabbat Chafjiri et al. [28] likewise identify training cost and data dependency as barriers to adoption. Within the learning-based category, LLM-guided fuzzing is the newest and fastest-emerging subcategory (predominantly 2024 onward), by recency rather than by count.

5.5. Mutation and Seed Generation Strategies

Across surveyed tools, mutation is built from six primitive mutation operators: argument mutation, function mutation, insertion/deletion, reorder, splice, and environment mutation (sender, timestamp, gas). Most tools apply ABI-aware random or coverage-directed variants of these; Cao et al. [86] find that the choice and stacking depth of Havoc-style mutators strongly affect coverage gains. Which mutations receive the most effort is governed by a power schedule (AFL-style energy assignment), adapted by MuFuzz and Vulseye but still underexplored at the sequence level. Beyond random and coverage-directed mutation, metaheuristic search has also been applied: EvoFuzzer [78] uses a genetic algorithm to evolve transaction sequences for reentrancy detection, and ACOFuzz [83] uses ant colony optimisation over the control-flow graph to guide exploration. Seed generation, finally, ranges from ABI/grammar-based (sFuzz, Echidna) to static-analysis-guided (DepFuzz [81], Erinys [57]), on-chain replay (ItyFuzz, Midas), and learning-based (LLM seeds [61]).

The tool genealogy in Figure 4 reveals two main lineages. The dominant one runs from ContractFuzzer (2018) through sFuzz and Echidna (2020) then ConFuzzius and SMARTIAN (2021) to the most recent generations of ItyFuzz, sFuzz2.0, and Vulseye (2025), progressively enriching coverage-guided fuzzing with symbolic, data-flow, and stateful feedback. A second, shorter lineage branches from Echidna toward the reinforcement-learning- and LLM-guided tools. Across both, each generation tends to add a single new feedback domain rather than compose existing ones, which confirms the composition gap identified in Section 4.2.4.

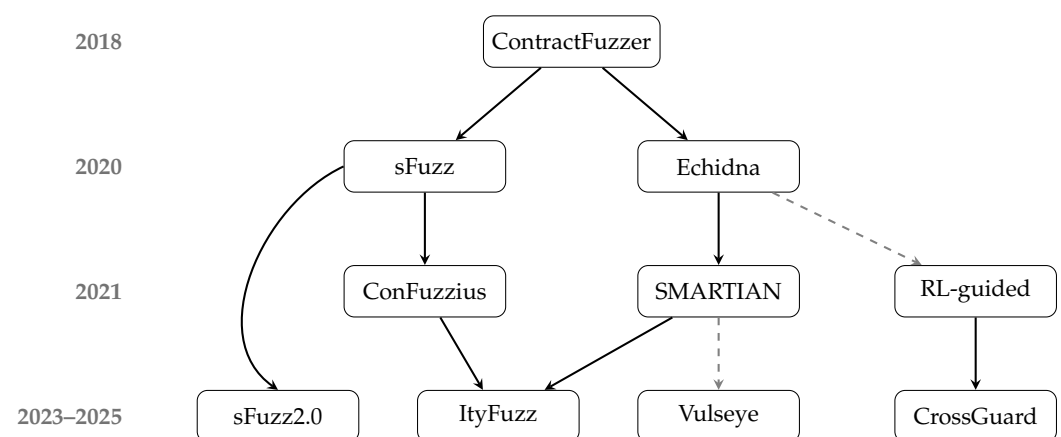


Figure 4. Simplified tool genealogy of representative smart contract fuzzers. Solid arrows denote direct lineage; dashed arrows denote conceptual influence.

5.6. Differential and Platform-Specific Fuzzing

Several further directions broaden smart contract fuzzing beyond single-contract vulnerability discovery. Differential fuzzers compare implementations and flag divergent behaviour as a bug: EVMFuzzer [87] across EVM implementations, Tian et al. [88] across Solidity compiler versions, and Fluffy and LOKI [89,90] across blockchain-client implementations. Platform-specific fuzzers port these techniques beyond Ethereum: EOSFuzzer [70] and WASAIUP [53] target EOSIO, Fuzz on the Beach [91] targets Solana, HFContractFuzzer [92] targets Hyperledger Fabric, and Hochrainer et al. [93] target zero-knowledge circuits. Cross-contract fuzzers such as CrossFuzz [58], xFuzz [76], and Pluto [71] reason about inter-contract interactions. Finally, exploit-generation tools synthesise working attacks rather than merely flagging vulnerabilities: EthPloit [69] and EF ∇ CF [94] generate exploits directly, and Verite [95] extracts over \$18M across 61 real-world DeFi exploits through profit-directed mutation.

5.7. Technique Comparison

The categories occupy distinct points in the effectiveness–cost trade-off. Coverage-guided fuzzing offers the highest throughput and the simplest deployment, but its coverage plateaus on branches guarded by specific values or reachable only through long transaction sequences. Property-based fuzzing provides the strongest, semantically meaningful oracles (user invariants), at the cost of manual specification effort. Hybrid symbolic/concolic fuzzing reaches guard-protected branches that pure mutation cannot, but inherits symbolic execution’s path explosion and solver cost. Learning-based fuzzing can improve seed quality and mutation/policy guidance, but its reproducibility depends on whether the underlying model is open or closed, and it is the least mature category. Differential and platform-specific fuzzing extend testing to cross-implementation and non-Ethereum targets, but require multiple reference implementations or platform-specific harnesses. No single category dominates across benchmarks [21], which is why tool portfolios combining coverage-guided, hybrid, and property-based fuzzers are the most effective practical strategy (Section 6.3). Table 6 draws these categories together, comparing 22 representative tools along five axes: input model, coverage type, oracle, hybrid or ML integration, and key innovation. Two trends stand out. First, almost all tools now operate on transaction sequences rather than single transactions, reflecting the stateful nature of contract execution. Second, most rely on plain branch coverage together with a reentrancy and arithmetic-oriented oracle suite; richer feedback (edge, storage-access, or property-level coverage) and hybrid or ML integration remain the exception rather than the rule, mirroring the composition gap identified in Section 4.2.4.

6. Discussion

6.1. Corpus Composition and Publication Trends

The 258 included items comprise 121 smart contract fuzzers, 31 web3 fuzzing surveys, 96 classical fuzzing surveys, 7 classical fuzzing tools, and 3 other works. The field has grown rapidly, from three SC-fuzzer papers in 2018 to 34 in 2024 (Figure 5), and is maturing, with 30.6% of papers appearing in Tier 1 venues (Figure 6). By technique (Figure 7), coverage-guided greybox fuzzing is the most common single technique (51.2%), followed by hybrid (24.0%), learning-based (10.7%), and differential (9.1%), with property-based (3.3%) and other approaches (1.7%) least represented. By platform, Ethereum/EVM accounts for 83.5% of tools, followed by other chains such as Hyperledger Fabric, WebAssembly, Solana, and multi-platform targets (11.6%), with EOSIO at 5.0%. Among the representative tools with detailed oracle information (Table 7), reentrancy is the most frequently targeted

vulnerability class (71%), followed by arithmetic (61%) and access control (48%), while financial properties remain comparatively under-served (<10%).

Table 7 cross-tabulates technique category against vulnerability class, and several patterns emerge. Reentrancy and arithmetic are targeted across all fuzzing-oriented categories, and hybrid fuzzers provide the broadest per-tool coverage despite being only 24.0% of tools. Financial properties, by contrast, are almost exclusively addressed by property-based fuzzers, while front-running/TOD and compiler bugs remain severely under-served (≤ 2 tools each).

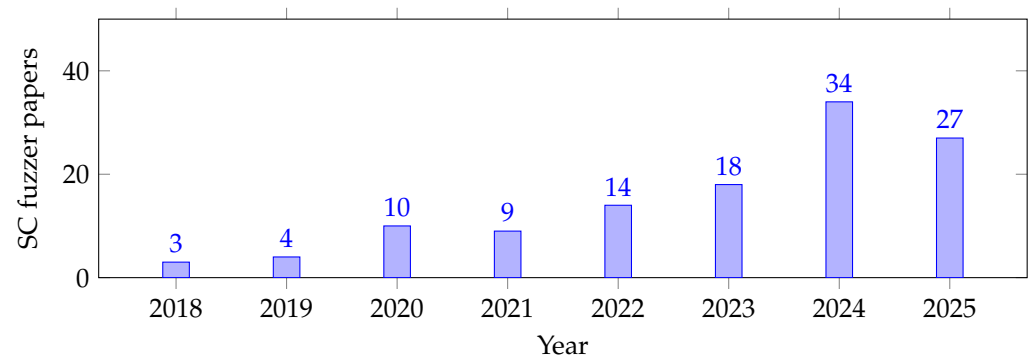


Figure 5. Year-by-year publication count for smart contract fuzzer papers in our corpus (119 SC fuzzers with a known year shown for 2018–2025; a further two papers dated 2026 are omitted; 2025 is partial).

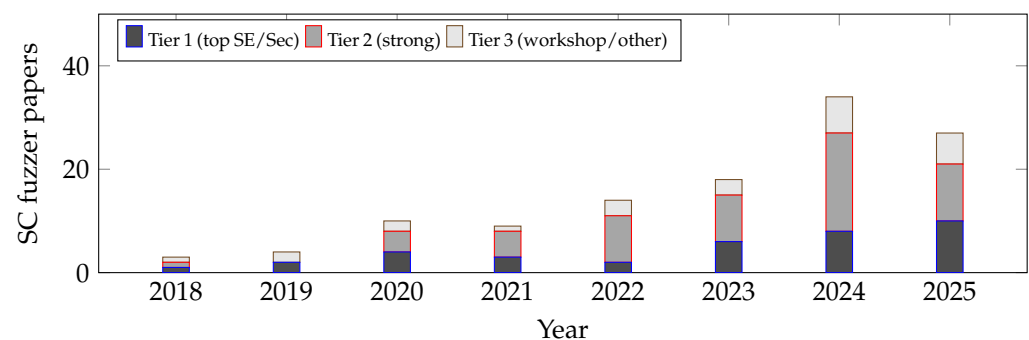


Figure 6. Venue-tier distribution (119 SC fuzzers with a classifiable venue shown for 2018–2025; two further 2026 tools omitted; full census $n = 121$): 30.6% Tier 1, 48.8% Tier 2, 20.7% Tier 3.

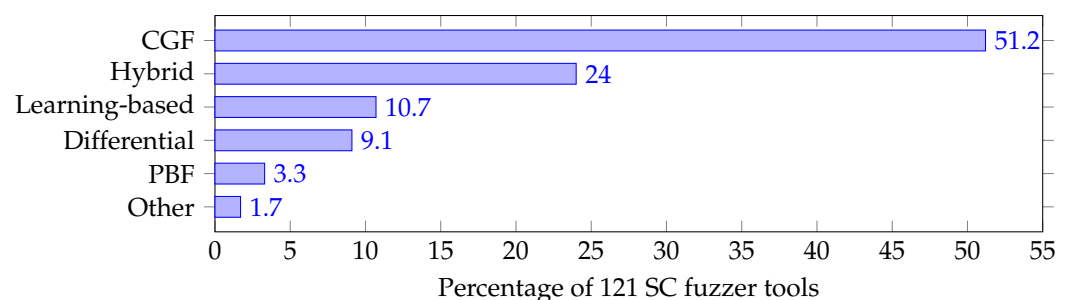


Figure 7. Technique distribution among 121 SC fuzzer tools. CGF is the most common technique (51.2%); learning-based is the fastest-emerging (LLM-guided, mostly post-2023).

Table 7. Technique category $\times$ vulnerability class cross-tabulation (31 of the 33 representative fuzzers; Olympia and GasFuzzer are excluded as they are a benchmark and a single-objective tool, respectively). R = reentrancy, AC = access control, Ar = arithmetic, G = gas, CC = cross-contract, Fin = financial, TOD = front-running.

Technique	R	AC	Ar	G	CC	Fin	TOD
CGF ($n = 20$)	16	10	14	4	4	1	1
PBF ($n = 3$)	2	1	1	0	0	2	0
Hybrid ($n = 5$)	4	4	4	0	1	0	0
Learning-based ($n = 2$)	0	0	0	0	2	0	0
Diff ($n = 1$)	0	0	0	0	0	0	0
Total	22	15	19	4	7	3	1

6.2. Answers to Research Questions

RQ1. Classical limitations (coverage bottlenecks, oracle design beyond crashes, scalability, ML cost) carry over to SC fuzzing and are amplified. The three challenges of semantic validity, stateful exploration, and environment modelling [25,47] all manifest simultaneously in the SC setting.

RQ2. The input is a transaction sequence (not a byte stream); execution is stateful; and oracles must inspect multi-transaction traces. These three structural differences help account for the shift from random to coverage-guided to hybrid approaches. Jain et al. [54] confirm that branch coverage is insufficient: high coverage does not imply high mutation score.

RQ3. EVM-specific constraints (256-bit words, gas metering, DELEGATECALL proxy patterns, ABI encoding, and compiler-version sensitivity) help account for the 83.5% Ethereum concentration. Tools requiring source code (Echidna, Foundry) cannot analyse bytecode-only contracts; DeepInfer [96] partially bridges this via deep learning-type recovery.

RQ4. Feedback-domain composition is sparse—the large majority of tools compose at most two of the eight domains and none approach the full eight (Section 4.2.4)—leaving several high-value multi-domain compositions unexplored. Property-based testing, grammar-based fuzzing, and dictionary mutation are underexplored (<5% each). SC innovations (stateful directed fuzzing, domain-specific oracles) can transfer to classical fuzzing of stateful systems [33].

6.3. Evaluation Methodology and Real-World Impact

No universally adopted benchmark exists: evaluation corpora range from hand-collected contract sets and the SmartBugs dataset to real-world DeFi protocols, and Wu et al. [21] demonstrate that the same fuzzer can appear effective or ineffective depending on which is chosen. Statistical rigour is also limited: significance testing is only rarely reported. Five recurring anti-patterns undermine comparability: (1) tuned-vs-default comparison; (2) cherry-picked contracts; (3) coverage-only evaluation; (4) missing baselines from the same technique category; and (5) single-run results without statistical analysis [97]. These five are recurring patterns illustrated by examples, identified during full-text reading, not an exhaustive per-paper census, and we do not use them to weight the self-reported results of individual papers. Reproducibility is uneven: while a number of tools release their source code or a public repository, many are closed-source or do not disclose availability at all (the per-tool status is released in the Availability field of the accompanying dataset). More importantly, the finer artefacts that genuine re-execution requires—an exact commit or version, build instructions, a pinned dependency specification, the seeds and fuzzing configurations used, the evaluation hardware, and an archivally persistent deposit—are essentially never reported in the surveyed papers, so their systematic absence, rather than

tool-by-tool variation, is the reproducibility finding. Even among the open-source tools, a reader typically cannot recover the exact configuration that produced the published numbers, which is why independent studies repeatedly re-implement baselines rather than reuse published artefacts.

In practice, Echidna [43] is a widely used property-based SC fuzzer, ItyFuzz adds on-chain state loading for triaging deployed contracts, and Foundry's Forge integrates fuzzing into the Solidity development workflow. Independent evaluations [6,21] consistently find that no single tool dominates: hybrid tools achieve the broadest vulnerability-class coverage, while CGF tools achieve the highest throughput. In practice, therefore, fuzzer portfolios running Echidna, ItyFuzz, and SMARTIAN in parallel emerge as the most practical deployment strategy.

6.4. Open Challenges and Lessons

Four open challenges stand out for smart contract fuzzing;

- Scalable stateful exploration. The input space grows as $(n \cdot m)^k$; hybrid and snapshot-based approaches partially address this, but deep ($k > 20$) stateful bugs remain hard to reach;
- Oracle design. Financial properties, front-running, and governance attacks are targeted by fewer than 10% of tools, and LLM-assisted invariant generation may help close this gap;
- EVM specifics. Gas metering, DELEGATECALL, ABI complexity, and compiler bugs all require EVM-aware tooling;
- Benchmarking. Curated, labelled benchmarks; multi-dimensional metrics (coverage, mutation score, and false-positive rate); and statistical testing are all urgently needed.

From these challenges, several practical lessons emerge for tool builders:

- Adopt sequence-aware mutation from the start, since single-transaction fuzzers miss stateful bugs entirely;
- Combine branch coverage with at least one state-aware signal, such as storage-access tracking or state hashing;
- Use demand-driven symbolic invocation rather than continuous concolic execution to control cost;
- Investing in oracle design is plausibly a high-value improvement for effective vulnerability detection;
- Release source code, benchmark contracts, and reproducibility scripts, as tools without artefacts have systematically lower research impact.

6.5. Threats to Validity

Several threats to validity should be noted, together with the steps taken to limit them. Search completeness is an external threat: to reduce the risk of missing relevant work, we queried two complementary databases (Scopus and Google Scholar) with different indexing scopes and screened all records following the PRISMA 2020 protocol. We nonetheless queried only these two indices, so works indexed in neither may have been missed, and because the Google Scholar component relies on the allintitle operator, which matches title terms only and runs against a non-deterministic, localisation-sensitive ranking, in-scope works whose titles omit our query terms may have been under-retrieved and exact result sets may not be reproducible. We mitigate the latter by releasing the raw Scholar export (all_scholar.csv), so the retained records remain verifiable even where the live query is not. Classification subjectivity is an internal threat, which we mitigated by applying a fixed coding scheme with explicit category definitions and cross-checking each classification against the tool's primary source. Several further threats we note explicitly. The corpus is

restricted to works with a full text in English (EC3), which may omit relevant non-English studies. The literature is subject to publication bias toward positive tool evaluations and to incremental republication of the same system across venues, and our headline figures rely on authors' self-reported results, which we do not independently re-run. We apply no formal per-paper study-quality weighting: quality is treated only as context through the venue tier, evaluation anti-patterns, and open-source availability. The 33 representative tools are chosen by purposive maximum-variation sampling and are therefore not a random sample. Distinguishing a tool from the paper that describes it is occasionally ambiguous for multi-paper systems, which we resolve by collapsing duplicate editions in the released dataset. The corpus is also strongly EVM/Ethereum-oriented, which we report as an empirical property of the field rather than a scope restriction, and findings on LLM-guided fuzzing may date quickly given the pace of that subfield. Finally, because indexing lags behind research activity, papers appearing close to our final search date of 15 February 2026 may be under-represented, a cut-off we state explicitly so that the coverage of the survey can be interpreted accordingly.

7. Conclusions

This survey reviewed the recent advances in smart contract fuzzing with a focus on methods, techniques, and architectures, organised around four research questions that link classical and smart contract-specific fuzzing. We synthesise our findings across the four research questions (answered in detail in the Discussion, Section 6) and close with an outlook.

The general fuzzing limitations of RQ1 all carry over to, and are amplified in, the smart contract setting. They comprise coverage bottlenecks, oracle design beyond crash detection, scalability, and the cost and interpretability of ML/LLM guidance, together with the challenges of semantic validity, stateful exploration, and environment modelling [25,47]. Here, the stateless execution model, byte-level input space, and crash-centric oracle of classical fuzzing must each be generalised (Table 3). Two further factors shape the landscape. At the domain level (RQ2), smart contracts differ from classical targets in three main ways: their inputs are transaction sequences, their execution is stateful, and their oracles must inspect execution traces. At the platform level (RQ3), EVM-specific constraints add further demands, including 256-bit words, gas metering, DELEGATECALL proxy patterns, ABI encoding, and compiler-version sensitivity. Together, these factors are consistent with the prevalence of coverage-guided greybox fuzzing (51.2% of the 121), the 83.5% concentration on Ethereum/EVM targets, and the under-service of cross-contract and financial (<10%) vulnerability classes. For transferability (RQ4), generalised waypoint mapping (Table 5, Section 4.2.4) gives the most concrete result: feedback-domain composition is sparse—the large majority of smart contract fuzzers compose at most two of the eight domains and none approach the full eight—leaving several high-value multi-domain compositions unexplored, since adding domains can only enlarge the retained set, never shrink it [45] (a formal property, not a demonstrated detection gain). Grammar-based and property-based fuzzing, dictionary and structure-aware mutation, and LLM/RL-guided generation are mature in classical fuzzing but still evolving for smart contracts, whereas transaction-sequence modelling, stateful directed fuzzing, and domain-specific oracles can flow back to classical fuzzing of stateful and protocol-oriented systems [31,33].

Beyond the per-RQ findings, our principal contributions fall into three groups. The first is a set of conceptual artefacts: a classical-to-smart contract fuzzing taxonomy (Table 3), a technique–vulnerability cross-tabulation (Table 7), and a generalised waypoint architecture (Table 5) that maps all 33 representative fuzzers to eight composable feedback domains and identifies several structural composition gaps (Section 4.2.4). The second is a set

of descriptive analyses of the field: classification and comparison tables (Tables 4–6), a technique genealogy (Figure 4), bibliometric analyses (Figures 5–7), and an evaluation-methodology analysis of five recurring anti-patterns. The third is a forward-looking research agenda (Section 7).

Looking forward, we identify three overarching priorities for the field:

1. Standardisation—shared benchmarks with ground-truth labels, multi-dimensional adequacy metrics, and mandatory artefact release;
2. Integration—tighter coupling of fuzzing with formal verification [7], specification mining, and LLM-assisted invariant generation, extending from contract-level to protocol-level DeFi properties [85];
3. Generalisation—extending smart contract fuzzing techniques to cross-chain protocols, layer-2 rollups, zero-knowledge circuits, and non-Ethereum platforms.

We highlight that smart contract fuzzing, with its inherently stateful execution model and the economic imperative of protecting over \$100 billion in DeFi TVL, offers both a critical application domain and a rich source of innovations that can advance the broader fuzzing field.

Supplementary Materials: The following supporting information can be downloaded at <https://www.mdpi.com/article/10.3390/app16189106/s1>: README.md: dataset description, population definitions, file inventory, and column dictionary; final_classification.csv: the umbrella corpus of all 258 included publications with their inclusion stratum; sc_fuzzer_core.csv: per-tool extraction for the 121 smart contract fuzzers (118 unique) forming the analytical core, with technique, feedback domain, input model, platform, oracles, availability, and subset-membership flags; scopus_raw.csv: raw Scopus export (647 records); all_scholar.csv: raw Google Scholar export (504 records); venue_tiers.csv: venue-tier assignment for all 121 core tools; Supplementary_S1_Search_Strategy.md: verbatim Boolean queries for both databases and reproducibility notes; Supplementary_S2_Coding_Manual.md: coding manual with eligibility criteria, the fuzzing-loop rule, decision rules, and the primary-category rule. The same package is archived on Zenodo (DOI: 10.5281/zenodo.22539912).

Author Contributions: Conceptualization, L.A.L.A., L.d.I.T. and Z.W.; methodology, L.A.L.A., L.d.I.T. and Z.W.; investigation, L.A.L.A.; writing—original draft preparation, L.A.L.A.; writing—review and editing, L.A.L.A., L.d.I.T. and Z.W.; supervision, L.d.I.T., Z.W. and S.D.C.; project administration, L.d.I.T. and S.D.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The complete corpus, per-record classification, extraction sheet, coding manual, and search strategy are openly available at Zenodo (DOI: 10.5281/zenodo.22539912) and as Supplementary Materials.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. Self-Published Whitepaper, 2008. Available online: <https://bitcoin.org/bitcoin.pdf> (accessed on 1 September 2025).
2. Wood, G. Ethereum: A Secure Decentralised Generalised Transaction Ledger. Ethereum Project Yellow Paper, 2014. Available online: <https://ethereum.github.io/yellowpaper/paper.pdf> (accessed on 7 September 2025).
3. Szabo, N. Formalizing and Securing Relationships on Public Networks. *First Monday* **1997**, 2. [CrossRef]
4. Ethereum Foundation. Solidity Documentation. 2023. Available online: <https://docs.soliditylang.org/> (accessed on 18 November 2025).

5. Zhu, H.; Yang, L.; Wang, L.; Sheng, V.S. A Survey on Security Analysis Methods of Smart Contracts. *IEEE Trans. Serv. Comput.* **2024**, *17*, 4522–4539. [CrossRef]
6. Hejazi, N.; Lashkari, A.H. A Comprehensive Survey of Smart Contracts Vulnerability Detection Tools: Techniques and Methodologies. *J. Netw. Comput. Appl.* **2025**, *237*, 104142. [CrossRef]
7. Tolmach, P.; Li, Y.; Lin, S.W.; Liu, Y.; Li, Z. A Survey of Smart Contract Formal Specification and Verification. *ACM Comput. Surv.* **2021**, *54*, 1–38. [CrossRef]
8. Trail of Bits. Slither: A Static Analysis Framework for Smart Contracts. 2018. Available online: <https://github.com/crytic/slither> (accessed on 24 January 2025).
9. Luu, L.; Chu, D.H.; Olickel, H.; Saxena, P.; Hobor, A. Making Smart Contracts Smarter. In Proceedings of the 23rd ACM Conference on Computer and Communications Security (CCS), Vienna, Austria, 24–28 October 2016; pp. 254–269. [CrossRef]
10. ConsenSys. Mythril: Security Analysis Tool for EVM Bytecode. 2017. Available online: <https://github.com/ConsenSys/mythril> (accessed on 9 March 2025).
11. Miller, B.P.; Fredriksen, L.; So, B. An Empirical Study of the Reliability of UNIX Utilities. *Commun. ACM* **1990**, *33*, 32–44. [CrossRef]
12. Zalewski, M. American Fuzzy Lop (AFL). 2015. Available online: <https://lcamtuf.coredump.cx/afl/> (accessed on 23 February 2025).
13. Serebryany, K. LibFuzzer: A Library for Coverage-Guided Fuzz Testing. 2015. Available online: <https://lvm.org/docs/LibFuzzer.html> (accessed on 23 February 2025).
14. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [CrossRef]
15. Kitchenham, B.; Charters, S. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*; Technical Report EBSE-2007-01; Keele University and Durham University: Staffordshire, UK, 2007.
16. Petersen, K.; Vakkalanka, S.; Kuzniarz, L. Guidelines for Conducting Systematic Mapping Studies in Software Engineering: An Update. *Inf. Softw. Technol.* **2015**, *64*, 1–18. [CrossRef]
17. Wei, Z.; Sun, J.; Zhang, Z.; Zhang, X.; Yang, X.; Zhu, L. Survey on Quality Assurance of Smart Contracts. *ACM Comput. Surv.* **2025**, *57*, 32. [CrossRef]
18. Dong, W.L.; Liu, Z.; Liu, K.; Li, L.; Ge, C.P.; Huang, Z.Q. Survey on Vulnerability Detection Technology of Smart Contracts. *Ruan Jian Xue Bao/J. Softw.* **2024**, *35*, 38–62. [CrossRef]
19. Qian, P.; Liu, Z.; He, Q.; Huang, B.; Tian, D.; Wang, X. Smart Contract Vulnerability Detection Technique: A Survey. *arXiv* **2022**, arXiv:2209.05872.
20. Kushwaha, S.S.; Joshi, S.; Singh, D.; Kaur, M.; Lee, H.N. Ethereum Smart Contract Analysis Tools: A Systematic Review. *IEEE Access* **2022**, *10*, 57037–57062. [CrossRef]
21. Wu, S.; Li, Z.; Yan, L.; Chen, W.; Jiang, M.; Wang, C.; Luo, X.; Zhou, H. Are We There Yet? Unraveling the State-of-the-Art Smart Contract Fuzzers. In Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, Lisbon, Portugal, 14–20 April 2024. [CrossRef]
22. Andrijasa, M.F.; Ismail, S.A.; Ahmad, N.; Yusop, O.M. Enhancing Smart Contract Security Through Multi-Agent Deep Reinforcement Learning Fuzzing: A Survey of Approaches and Techniques. *Int. J. Adv. Comput. Sci. Appl.* **2024**, *15*, 754–767. [CrossRef]
23. Mnasri, M.; Imal Maâlej, A.; Jmaiel, M. Comprehensive Overview: Ethereum Smart Contract Security Testing for Reentrancy Issues. In *Proceedings of the Service-Oriented Computing—ICSOC 2024 Workshops*; Springer: Singapore, 2026; pp. 202–207. [CrossRef]
24. Manès, V.J.; Han, H.; Han, C.; Cha, S.K.; Egele, M.; Schwartz, E.J.; Woo, M. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Trans. Softw. Eng.* **2021**, *47*, 2312–2331. [CrossRef]
25. Böhme, M.; Cadar, C.; Roychoudhury, A. Fuzzing: Challenges and Reflections. *IEEE Softw.* **2021**, *38*, 79–86. [CrossRef]
26. Beaman, C.; Redbourne, M.; Mummery, J.D.; Hakak, S. Fuzzing vulnerability discovery techniques: Survey, challenges and future directions. *Comput. Secur.* **2022**, *120*, 102813. [CrossRef]
27. Mallisery, S.; Wu, Y.S. Demystify the Fuzzing Methods: A Comprehensive Survey. *ACM Comput. Surv.* **2023**, *56*, 71:1–71:38. [CrossRef]
28. Bamohabbat Chafjiri, S.; Legg, P.; Hong, J.; Tsompanas, M.A. Vulnerability detection through machine learning-based fuzzing: A systematic review. *Comput. Secur.* **2024**, *143*, 103903. [CrossRef]
29. Qiu, J.; Jiang, Y.; Miao, Y.; Luo, W.; Pan, L.; Zheng, X. A survey of coverage-guided greybox fuzzing with deep neural models. *Inf. Softw. Technol.* **2025**, *186*, 107797. [CrossRef]
30. Dube, E.L.; Akinnuwesi, B.A.; Fashoto, S.G.; Mashwama, P.M.; Tsabedze, V.W. Improving greybox fuzzing with dictionary-based mutations: A systematic literature review. *S. Afr. Comput. J.* **2025**, *37*, 74–103. [CrossRef]
31. Zheng, C.; Wang, Y.; Huang, H.; Wang, Y.; Chen, H.; Wei, Q. Survey of network protocol fuzzers: Taxonomy, techniques, and directions. *Comput. Secur.* **2025**, *159*, 104683. [CrossRef]

32. Zhen, W.; Huang, M.; Li, X.; Nie, Y.; Chu, L.; Wan, Y. A Survey of Kernel Fuzzing. In *Proceedings of the Cyberspace Safety and Security*; Springer: Singapore, 2026; pp. 301–315. [\[CrossRef\]](#)
33. Daniele, C.; Andarzian, S.B.; Poll, E. Fuzzers for Stateful Systems: Survey and Research Directions. *ACM Comput. Surv.* **2024**, *56*, 222:1–222:23. [\[CrossRef\]](#)
34. Wan, Y.; Huang, M.; Li, X.; Nie, Y.; Zhen, W.; Zhong, F. Large language models-driven fuzzing: A systematic survey. *Eng. Appl. Artif. Intell.* **2026**, *178*, 115126. [\[CrossRef\]](#)
35. Zhang, A.; Zhang, Y.; Xu, Y.; Wang, C.; Li, S. Machine Learning-Based Fuzz Testing Techniques: A Survey. *IEEE Access* **2024**, *12*, 14437–14454. [\[CrossRef\]](#)
36. Google. OSS-Fuzz: Continuous Fuzzing for Open Source Software. 2016. Available online: <https://github.com/google/oss-fuzz> (accessed on 12 May 2025).
37. Swiecki, R. Honggfuzz: A Security Oriented Fuzzer. 2016. Available online: <https://github.com/google/honggfuzz> (accessed on 17 May 2025).
38. Böhme, M.; Pham, V.T.; Roychoudhury, A. Coverage-Based Greybox Fuzzing as Markov Chain. In *Proceedings of the 23rd ACM Conference on Computer and Communications Security (CCS)*, Vienna, Austria, 24–28 October 2016; pp. 1032–1043. [\[CrossRef\]](#)
39. Lyu, C.; Ji, S.; Zhang, C.; Li, Y.; Lee, W.H.; Song, Y.; Beyah, R. MOPT: Optimized Mutation Scheduling for Fuzzers. In *Proceedings of the 28th USENIX Security Symposium*, Santa Clara, CA, USA, 14–16 August 2019; pp. 1949–1966.
40. Lemieux, C.; Sen, K. FairFuzz: A Targeted Mutation Strategy for Increasing Greybox Fuzz Testing Coverage. In *Proceedings of the 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Montpellier, France, 3–7 September 2018; pp. 475–485. [\[CrossRef\]](#)
41. Nguyen, T.D.; Pham, L.H.; Sun, J.; Lin, Y.; Minh, Q.T. sFuzz: An efficient adaptive fuzzer for solidity smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, Seoul, Republic of Korea, 27 June–19 July 2020; pp. 778–788. [\[CrossRef\]](#)
42. Qian, P.; Wu, H.; Du, Z.; Vural, T.; Rong, D.; Cao, Z.; Zhang, L.; Wang, Y.; Chen, J.; He, Q. MuFuzz: Sequence-Aware Mutation and Seed Mask Guidance for Blockchain Smart Contract Fuzzing. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*, Utrecht, The Netherlands, 13–16 May 2024; pp. 1972–1985. [\[CrossRef\]](#)
43. Grieco, G.; Song, W.; Cygan, A.; Feist, J.; Groce, A. Echidna: Effective, usable, and fast fuzzing for smart contracts. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, Virtual Event, 18–22 July 2020; pp. 557–560. [\[CrossRef\]](#)
44. Shou, C.; Tan, S.; Sen, K. ItyFuzz: Snapshot-Based Fuzzer for Smart Contract. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, Seattle, WA, USA, 17–21 July 2023; pp. 322–333. [\[CrossRef\]](#)
45. Padhye, R.; Lemieux, C.; Sen, K.; Simon, L.; Vijayakumar, H. FuzzFactory: Domain-Specific Fuzzing with Waypoints. In *Proceedings of the ACM on Programming Languages (OOPSLA)*; Association for Computing Machinery: New York, NY, USA, 2019; Volume 3. [\[CrossRef\]](#)
46. Wang, H.; Wang, Z.; Liu, S.; Sun, J.; Zhao, Y.; Wan, Y.; Nguyen, T.D. sFuzz2.0: Storage-access pattern guided smart contract fuzzing. *J. Softw. Evol. Process* **2024**, *36*, e2557. [\[CrossRef\]](#)
47. Torres, C.F.; Iannillo, A.K.; Gervais, A.; State, R. ConFuzzius: A Data Dependency-Aware Hybrid Fuzzer for Smart Contracts. In *Proceedings of the 2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, Vienna, Austria, 6–10 September 2021; pp. 103–119. [\[CrossRef\]](#)
48. Claessen, K.; Hughes, J. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*; Association for Computing Machinery: New York, NY, USA, 2000; pp. 268–279. [\[CrossRef\]](#)
49. Cadar, C.; Dunbar, D.; Engler, D. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, San Diego, CA, USA, 8–10 December 2008; pp. 209–224.
50. Godefroid, P.; Levin, M.Y.; Molnar, D. Automated Whitebox Fuzz Testing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 10–13 February 2008.
51. Stephens, N.; Grosen, J.; Salls, C.; Dutcher, A.; Wang, R.; Corbetta, J.; Shoshitaishvili, Y.; Kruegel, C.; Vigna, G. Driller: Augmenting Fuzzing Through Selective Symbolic Execution. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 21–24 February 2016. [\[CrossRef\]](#)
52. Shi, F.; Yang, J.; Guo, Z. TPH-Fuzz: A Two-Phase Hybrid Fuzzing Framework for Smart Contract Vulnerability Detection. *Electronics* **2025**, *14*, 1465. [\[CrossRef\]](#)
53. Wang, M.; Wang, Z.; Yu, B.; Zhang, K. WASAIUP: A Demand-driven Concolic Fuzzer for EOSIO Smart Contracts. In *Proceedings of the 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*, Chiang Mai, Thailand, 22–26 October 2023; pp. 150–161. [\[CrossRef\]](#)

54. Jain, K.; Kalburgi, G.T.; Le Goues, C.; Groce, A. Mind the Gap: The Difference Between Coverage and Mutation Score Can Guide Testing Efforts. In Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy, 9–12 October 2023; pp. 102–113. [\[CrossRef\]](#)
55. Wong, T.; Zhang, C.; Ni, Y.; Luo, M.; Chen, H.; Yu, Y.; Li, W.; Luo, X.; Wang, H. ConFuzz: Towards Large Scale Fuzz Testing of Smart Contracts in Ethereum. In Proceedings of the IEEE INFOCOM 2024—IEEE Conference on Computer Communications, Vancouver, BC, Canada, 20–23 May 2024; pp. 1691–1700. [\[CrossRef\]](#)
56. Ye, M.; Lin, X.; Nan, Y.; Wu, J.; Zheng, Z. Midas: Mining Profitable Exploits in On-Chain Smart Contracts via Feedback-Driven Fuzzing and Differential Analysis. In Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, Vienna, Austria, 16–20 September 2024; pp. 794–805. [\[CrossRef\]](#)
57. Dong, C.; Huang, H.; Shang, Y. Erinys: Efficient fuzzing by function invoke sequence generation for smart contracts. In Proceedings of the 2024 8th International Conference on Big Data and Internet of Things, Macau, China, 14–16 September 2024; pp. 236–241. [\[CrossRef\]](#)
58. Yang, H.; Gu, X.; Chen, X.; Zheng, L.; Cui, Z. CrossFuzz: Cross-contract fuzzing for smart contract vulnerability detection. *Sci. Comput. Program.* **2024**, *234*, 103076. [\[CrossRef\]](#)
59. Zhang, J.; Zheng, Z.; Nan, Y.; Ye, M.; Ning, K.; Zhang, Y.; Zhang, W. Smartreco: Detecting Read-Only Reentrancy via Fine-Grained Cross-DApp Analysis. In Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 26 April–6 May 2025; pp. 2138–2150. [\[CrossRef\]](#)
60. Gan, J.; Su, J.; Lin, K.; Zheng, Z. FinanceFuzz: Fuzzing smart contracts with financial properties. *Blockchain Res. Appl.* **2025**, *6*, 100301. [\[CrossRef\]](#)
61. Sun, J.; Yin, Z.; Zhang, H.; Chen, X.; Zheng, W. Adversarial generation method for smart contract fuzz testing seeds guided by chain-based LLM. *Autom. Softw. Eng.* **2024**, *32*, 12. [\[CrossRef\]](#)
62. Su, J.; Dai, H.N.; Zhao, L.; Zheng, Z.; Luo, X. Effectively Generating Vulnerable Transaction Sequences in Smart Contracts with Reinforcement Learning-guided Fuzzing. In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE), Rochester, MI, USA, 10–14 October 2022; pp. 1–12. [\[CrossRef\]](#)
63. Jiang, B.; Liu, Y.; Chan, W.K. ContractFuzzer: Fuzzing smart contracts for vulnerability detection. In Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, Montpellier, France, 3–7 September 2018; pp. 259–269. [\[CrossRef\]](#)
64. Choi, J.; Kim, D.; Kim, S.; Grieco, G.; Groce, A.; Cha, S.K. SMARTIAN: Enhancing Smart Contract Fuzzing with Static and Dynamic Data-Flow Analyses. In Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), Melbourne, Australia, 15–19 November 2021; pp. 227–239. [\[CrossRef\]](#)
65. Yang, J.; Zhao, X.; Zhang, H.; He, L.; Wang, S.; Gou, N. CSAFuzzer: Fuzzing smart contracts combining with static analysis. *Empir. Softw. Eng.* **2025**, *30*, 62. [\[CrossRef\]](#)
66. Chan, W.; Jiang, B. Fuse: An Architecture for Smart Contract Fuzz Testing Service. In Proceedings of the 2018 25th Asia-Pacific Software Engineering Conference (APSEC), Nara, Japan, 4–7 December 2018; pp. 707–708. [\[CrossRef\]](#)
67. Wüstholtz, V.; Christakis, M. Harvey: A Greybox Fuzzer for Smart Contracts. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Virtual Event, 8–13 November 2020; pp. 1398–1409. [\[CrossRef\]](#)
68. Ashraf, I.; Ma, X.; Jiang, B.; Chan, W.K. GasFuzzer: Fuzzing Ethereum Smart Contract Binaries to Expose Gas-Oriented Exception Security Vulnerabilities. *IEEE Access* **2020**, *8*, 99552–99564. [\[CrossRef\]](#)
69. Zhang, Q.; Wang, Y.; Li, J.; Ma, S. EthPloit: From Fuzzing to Efficient Exploit Generation against Smart Contracts. In Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), London, ON, Canada, 18–21 February 2020; pp. 116–126. [\[CrossRef\]](#)
70. Huang, Y.; Bian, H.; Li, R.; Shi, Z. EOSFuzzer: Fuzzing EOSIO Smart Contracts for Vulnerability Detection. In Proceedings of the 12th Asia-Pacific Symposium on Internetware (Internetware), Singapore, 1–3 November 2020; pp. 99–109. [\[CrossRef\]](#)
71. Ma, F.; Xu, Z.; Ren, M.; Yin, Z.; Chen, Y.; Qiao, L.; Gu, B.; Li, H.; Jiang, Y.; Sun, J. Pluto: Exposing Vulnerabilities in Inter-Contract Scenarios. *IEEE Trans. Softw. Eng.* **2022**, *48*, 4380–4396. [\[CrossRef\]](#)
72. Li, B.; Pan, Z.; Hu, T. ReDefender: Detecting Reentrancy Vulnerabilities in Smart Contracts Automatically. *IEEE Trans. Reliab.* **2022**, *71*, 984–999. [\[CrossRef\]](#)
73. Feng, H.; Ren, X.; Wei, Q.; Lei, Y.; Kacker, R.; Kuhn, D.R.; Simos, D.E. MagicMirror: Towards High-Coverage Fuzzing of Smart Contracts. In Proceedings of the 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), Dublin, Ireland, 16–20 April 2023; pp. 141–152. [\[CrossRef\]](#)
74. Liao, J.; Hu, H.; Huang, K.; Jin, H.; Lin, C. LDfuzz: A Directed Greybox Fuzzer for Solidity Smart Contract. In Proceedings of the 2023 19th International Conference on Mobility, Sensing and Networking (MSN), Nanjing, China, 14–16 December 2023; pp. 488–495. [\[CrossRef\]](#)

75. Ji, S.; Wu, J.; Qiu, J.; Dong, J. Effuzz: Efficient fuzzing by directed search for smart contracts. *Inf. Softw. Technol.* **2023**, *159*, 107213. [\[CrossRef\]](#)
76. Xue, Y.; Ye, J.; Zhang, W.; Sun, J.; Ma, L.; Wang, H.; Zhao, J. xFuzz: Machine Learning Guided Cross-Contract Fuzzing. *IEEE Trans. Dependable Secur. Comput.* **2024**, *21*, 515–529. [\[CrossRef\]](#)
77. Ye, M.; Nan, Y.; Dai, H.N.; Yang, S.; Luo, X.; Zheng, Z. FunFuzz: A Function-Oriented Fuzzer for Smart Contract Vulnerability Detection with High Effectiveness and Efficiency. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 191:1–191:20. [\[CrossRef\]](#)
78. Li, B.; Pan, Z.; Hu, T. EvoFuzzer: An Evolutionary Fuzzer for Detecting Reentrancy Vulnerability in Smart Contracts. *IEEE Trans. Netw. Sci. Eng.* **2024**, *11*, 5790–5802. [\[CrossRef\]](#)
79. Chadt, J.; Hochrainer, C.; Wüstholtz, V.; Christakis, M. Olympia: Fuzzer Benchmarking for Solidity. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; pp. 2362–2365. [\[CrossRef\]](#)
80. Liang, R.; Chen, J.; Wu, C.; He, K.; Wu, Y.; Cao, R.; Du, R.; Zhao, Z.; Liu, Y. Vulseye: Detect Smart Contract Vulnerabilities via Stateful Directed Graybox Fuzzing. *IEEE Trans. Inf. Forensics Secur.* **2025**, *20*, 2157–2170. [\[CrossRef\]](#)
81. Ma, C.; Song, W.; Huang, J. DepFuzz: Efficient Smart Contract Fuzzing with Function Dependence Guidance. *Proc. ACM Program. Lang.* **2025**, *9*, 407:3755–407:3783. [\[CrossRef\]](#)
82. Ali, G.M.A.; Chen, H.; Wang, Z.; Du, C. CrossGuard: Runtime-Adaptive LLM Fuzzing for Cross-Contract Vulnerabilities Detection. *Concurr. Comput. Pract. Exp.* **2025**, *37*, e70421. [\[CrossRef\]](#)
83. Feng, P.; Cao, W.; Lu, S.; Wang, Y.; Xue, H.; Yang, R. ACOFuzz: An ant colony algorithm-based fuzzer for smart contracts. *Blockchain Res. Appl.* **2025**, *6*, 100279. [\[CrossRef\]](#)
84. Wesley, S.; Christakis, M.; Navas, J.A.; Trefler, R.; Wüstholtz, V.; Gurfinkel, A. Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE. In *Proceedings of the Verification, Model Checking, and Abstract Interpretation*; Springer: Cham, Switzerland, 2022; pp. 425–449. [\[CrossRef\]](#)
85. Ding, Y.; Gervais, A.; Wattenhofer, R.; Sato, H. Hunting DeFi Vulnerabilities via Context-Sensitive Concolic Verification. In Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, Lisbon, Portugal, 14–20 April 2024; pp. 324–325. [\[CrossRef\]](#)
86. Cao, J.; Wang, M.; Lin, S. Rethinking Mutation Strategies in Fuzzing Smart Contracts. In Proceedings of the 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Sanya, China, 17–21 December 2024; pp. 650–657. [\[CrossRef\]](#)
87. Fu, Y.; Ren, M.; Ma, F.; Shi, H.; Yang, X.; Jiang, Y.; Li, H.; Shi, X. EVMFuzzer: Detect EVM vulnerabilities via fuzz testing. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Tallinn, Estonia, 26–30 August 2019; pp. 1110–1114. [\[CrossRef\]](#)
88. Tian, Z.; Wang, F.; Chen, Y.; Chen, L. Differential testing solidity compiler through deep contract manipulation and mutation. *Softw. Qual. J.* **2024**, *32*, 765–790. [\[CrossRef\]](#)
89. Yang, Y.; Kim, T.; Chun, B.G. Finding Consensus Bugs in Ethereum via Multi-transaction Differential Fuzzing. In Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), Virtual Event, 14–16 July 2021; pp. 349–365.
90. Ma, F.; Chen, Y.; Ren, M.; Zhou, Y.; Jiang, Y.; Chen, T.; Li, H.; Sun, J. LOKI: State-Aware Fuzzing Framework for the Implementation of Blockchain Consensus Protocols. In Proceedings of the Network and Distributed System Security Symposium, San Diego, CA, USA, 27 February–3 March 2023. [\[CrossRef\]](#)
91. Smolka, S.; Giesen, J.R.; Winkler, P.; Draissi, O.; Davi, L.; Karame, G.; Pohl, K. Fuzz on the Beach: Fuzzing Solana Smart Contracts. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Copenhagen, Denmark, 26–30 November 2023; pp. 1197–1211. [\[CrossRef\]](#)
92. Ding, M.; Li, P.; Li, S.; Zhang, H. HFContractFuzzer: Fuzzing Hyperledger Fabric Smart Contracts for Vulnerability Detection. In Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering, Trondheim, Norway, 21–23 June 2021; pp. 321–328. [\[CrossRef\]](#)
93. Hochrainer, C.; Isychev, A.; Wüstholtz, V.; Christakis, M. Fuzzing Processing Pipelines for Zero-Knowledge Circuits. In Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, Taipei, Taiwan, 13–17 October 2025; pp. 783–797. [\[CrossRef\]](#)
94. Rodler, M.; Paaßen, D.; Li, W.; Bernhard, L.; Holz, T.; Karame, G.; Davi, L. EF/CF: High Performance Smart Contract Fuzzing for Exploit Generation. *arXiv* **2023**. [\[CrossRef\]](#)
95. Kong, Z.; Zhang, C.; Xie, M.; Hu, M.; Xue, Y.; Liu, Y.; Wang, H.; Liu, Y. Smart Contract Fuzzing Towards Profitable Vulnerabilities. *Proc. ACM Softw. Eng.* **2025**, *2*, 153–175. [\[CrossRef\]](#)

96. Zhao, K.; Li, Z.; Li, J.; Ye, H.; Luo, X.; Chen, T. DeepInfer: Deep Type Inference from Smart Contract Bytecode. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco, CA, USA, 3–9 December 2023; pp. 745–757. [[CrossRef](#)]
97. Arcuri, A.; Briand, L. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In Proceedings of the 33rd International Conference on Software Engineering (ICSE), Honolulu, HI, USA, 21–28 May 2011; pp. 1–10. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.